

EEE 468 (July 2025)

VLSI Laboratory

Final Project Report

Section: G2 Group: 03

Design and Physical Implementation of a 4×4 Wallace Tree Multiplier

Course Instructors:

Nafis Sadik, Assistant Professor

Archishman Sarkar, Part-Time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: Noor

Full Name: Golam An Noor Siddique

Student ID: 2006074

Signature: Anay

Full Name: Md Al Saif Hossain Anay

Student ID: 2006077

Signature: Nakib

Full Name: Md Nakib

Student ID: 2006091

Signature: Mostafizur

Full Name: Md Mostafizur Rahman

Student ID: 2006096

Signature: Samia

Full Name: Samia Hossain

Student ID: 2006138

Table of Contents

1	Abstract.....	1
2	Introduction	1
3	Design	2
3.1	Problem Formulation (PO(b)).....	2
3.1.1	Identification of Scope	2
3.1.2	Literature Review.....	2
3.1.3	Formulation of Problem.....	2
3.1.4	Analysis	2
3.2	Design Method (PO(a)).....	4
4	Implementation	7
4.1	RTL.....	7
4.2	Implementation of Algorithm.....	2
4.3	Directed Testbench	3
4.4	Layered Testbench	4
4.5	Synthesis	5
4.6	Physical Design	
5	Design Analysis and Evaluation	62
5.1	Novelty.....	62
5.2	Design Considerations (PO(c))	62
5.2.1	Considerations to public health and safety	62
5.2.2	Considerations to environment	62
5.2.3	Considerations to cultural and societal needs	62
5.3	Investigations (PO(d)).....	62
5.3.1	Design of Experiment	Error! Bookmark not defined.
5.3.2	Data Collection	Error! Bookmark not defined.
5.3.3	Results and Analysis	Error! Bookmark not defined.
5.3.4	Interpretation and Conclusions on Data.....	Error! Bookmark not defined.
5.4	Limitations of Tools (PO(e))	64
6	Reflection on Individual and Team work (PO(i)).....	64
6.1	Individual Contribution of Each Member.....	64
6.2	Mode of TeamWork.....	64
6.3	Diversity Statement of Team	64

7	Communication to External Stakeholders (PO(j))	65
7.1	Executive Summary	65
7.3	EDA code Link	65
8	Project Management and Cost Analysis (PO(k)).....	65
8.1	Timeline of Project Implementation	65
9	Future Work (PO(l)).....	66
10	References	66

1 Abstract

This project presents the design and physical implementation of a 4×4 Wallace tree multiplier using Verilog HDL. The multiplier takes two 4 bit input operands and produces an 8-bit output through parallel partial product generation and reduction. Compared to conventional sequential multiplication approaches, the Wallace tree structure reduces propagation delay by compressing partial products in parallel using half adders and full adders. To improve throughput, a pipelined architecture with 2 cycle latency is implemented. The functional correctness of the design is verified using both directed and layered SystemVerilog testbenches. Directed testing includes random and exhaustive input combinations, while the layered testbench verifies functional behavior, reset operation, register capture, pipeline latency, and input ordering. After verification, the design is synthesized and analyzed in terms of power and timing. Physical implementation is then carried out through floorplanning, placement, clock tree synthesis, routing, and DRC checking. The final design achieves correct operation with positive timing slack and successful layout implementation, demonstrating that the Wallace tree multiplier is an effective architecture for high speed and physically realizable digital multiplication.

2 Introduction

Multiplication is one of the most fundamental arithmetic operations in digital systems and is widely used in processors, arithmetic logic units, digital signal processing, and image processing applications. Since the multiplier often lies on the critical path of a system, its architecture has a direct impact on speed, area, and power consumption. Conventional multiplication methods such as repeated addition or array based multiplication are simple to implement, but they generally suffer from higher delay due to sequential partial product addition. To overcome this limitation, faster architectures such as the Wallace tree multiplier are used, where partial products are reduced in parallel using half adders and full adders.

In this project, a 4×4 Wallace tree multiplier has been designed and physically implemented using Verilog HDL. The design multiplies two 4 bit inputs to produce an 8 bit output and uses a pipelined structure to improve throughput. The project covers the complete VLSI design flow, including RTL design, directed and layered verification, synthesis, timing and power analysis, and physical implementation. The Wallace tree architecture is selected because it provides fast parallel reduction with a clear and practical implementation flow, making it suitable for a high-speed exact multiplier design.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The scope of this project is to design and physically implement an exact 4×4 Wallace tree multiplier using a complete digital VLSI design flow.

- Design of the 4×4 Wallace tree multiplier in Verilog HDL
- Use of a 2 cycle pipelined architecture for improved throughput
- Functional verification using directed and layered testbenches
- Synthesis and analysis of power and timing performance
- Physical implementation including floorplanning, placement, routing, and DRC checking

The work is limited to an exact 4×4 multiplier and does not include larger multiplier sizes or approximate multiplication methods.

3.1.2 Literature Review

Multiplication is one of the most important arithmetic operations in digital systems, and the choice of multiplier architecture significantly affects the overall speed, area, and power consumption of the design. Over the years, several multiplier structures have been proposed to improve these performance metrics for different application requirements.

One of the earliest important approaches is the Booth multiplier, introduced to reduce the number of partial products through recoding of the multiplier bits. This method is particularly useful for signed multiplication, since it handles positive and negative operands in a uniform manner. However, the recoding and control logic increase architectural complexity compared to simpler unsigned designs [1].

For signed parallel multiplication, the Baugh–Wooley multiplier provides a more regular hardware structure by rearranging two's complement partial products into a form suitable for array implementation. This regularity makes the architecture attractive for VLSI realization. Later comparative studies have also shown that Baugh–Wooley based designs can achieve competitive delay with lower power and area than modified Booth implementations in some process technologies [2].

To achieve higher speed, Wallace proposed a tree based partial product reduction technique in which the generated partial products are compressed in parallel rather than being added sequentially. This substantially reduces the reduction delay and makes Wallace multipliers suitable for high speed arithmetic circuits [3]. A refinement of this idea was later introduced by Dadda, who proposed that only the minimum required reduction should be performed at each stage. This leads to fewer counters and often a more area-efficient implementation while preserving the logarithmic reduction depth of tree multipliers [4]. Comparative analysis of column compression multipliers shows that Wallace multipliers generally occupy slightly more area, while Dadda multipliers achieve approximately similar worst case delay in deep submicron implementations [5].

More recent studies have focused on improving the internal building blocks used in tree multipliers. Janveja and Niranjana presented a Wallace tree multiplier using an improved adder and reported significant reductions in power dissipation, power delay product, and transistor count compared to an existing Wallace structure, indicating that the efficiency of Wallace multipliers can be further enhanced through optimized reduction stage circuitry [6]. Similarly, Ramanathan *et al.* proposed a modified Dadda multiplier using higher order compressors and showed that compressor based reduction can significantly reduce power delay product compared to conventional Dadda designs [7].

In addition, approximate tree compressor based multipliers have been proposed for error tolerant applications, where limited computational inaccuracy is acceptable in exchange for lower power, smaller area, and shorter critical path delay. Yang *et al.* showed that such approximate multipliers can achieve large reductions in power, delay, and area relative to a conventional Wallace multiplier, although these architectures are not suitable for exact arithmetic systems [8].

From the above review, it is evident that array based multipliers offer greater structural regularity, while tree based multipliers such as Wallace and Dadda provide superior speed due to parallel partial product reduction [3]–[5]. Since the objective of this project is to implement an exact 4×4 multiplier with high speed, clear RTL structure, and straightforward synthesis and physical design flow, the Wallace tree multiplier is selected as the most appropriate architecture.

Table: Comparison of Different Multiplier Architectures

Multiplier Architecture	Basic Principle	Main Advantages	Main Limitations	Typical Use
Booth Multiplier	Recodes multiplier bits to reduce partial products	Efficient for signed multiplication; fewer partial products	Extra recoding logic increases design complexity	Signed multiplication [1]
Baugh–Wooley Multiplier	Rearranges two's-complement partial products into regular array form	Regular structure; suitable for signed hardware implementation	Delay advantage is limited compared to tree-based multipliers	Signed parallel multiplication [2]
Wallace Tree Multiplier	Compresses partial products in parallel using reduction tree	Very high speed; low reduction delay	Less regular structure; may require more hardware	High-speed exact multiplication [3], [6]
Dadda Multiplier	Performs minimum required reduction at each stage	Lower number of counters; area-efficient	Reduction scheduling is more involved	Fast and area-conscious multiplication [4], [5], [7]
Approximate Tree Multiplier	Uses simplified compressor tree with controlled error	Low power, low area, shorter delay	Not suitable for exact arithmetic	Error-tolerant applications [8]

3.1.3 Formulation of Problem

The problem addressed in this project is the design and implementation of a high speed and physically realizable 4×4 multiplier using an efficient architecture. Conventional multiplication methods suffer from increased propagation delay due to sequential addition of partial products, which limits performance in time critical digital systems. Therefore, an alternative approach is required that can reduce delay while maintaining correctness and feasibility for hardware implementation.

The problem can be formulated as follows:

- To design a 4×4 multiplier that performs correct binary multiplication of two input operands
- To reduce propagation delay using parallel partial product reduction (Wallace tree structure)
- To incorporate a pipelined architecture for improved throughput and timing performance
- To ensure the design is functionally correct and fully verified under different operating conditions
- To implement the design through synthesis and physical layout, ensuring timing closure and design rule compliance

Thus, the core challenge is to develop a multiplier that is not only fast and accurate but also suitable for complete VLSI implementation from RTL to layout.

3.2 Design Method (PO(a))

The proposed multiplier is designed using the Wallace tree multiplication method, where multiplication is performed through three main steps: partial product generation, partial product reduction, and final addition. In this project, two 4 bit operands, $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$, are multiplied to generate an 8 bit output. The overall objective of the Wallace tree approach is to reduce the delay of multiplication by compressing partial products in parallel instead of adding them sequentially.

First, all partial products are generated by bitwise AND operations between the bits of the two operands. Thus, for a 4×4 multiplier, a total of 16 partial products are produced, namely $P_{ij} = A_iB_j$, where $i, j = 0, 1, 2, 3$. Figure 1 shows the conventional generation of partial products for the multiplication of two 4-bit numbers. In the lower part of the figure, these partial products are rearranged in a structured form by grouping them according to the $(i+j)$ column index. Since each partial product $P_{ij} = A_iB_j$ has a binary weight of 2^{i+j} , all partial products with the same value of $(i+j)$ belong to the same column. This arrangement is essential for Wallace tree reduction, because bits of equal weight can then be compressed together using half adders and full adders.

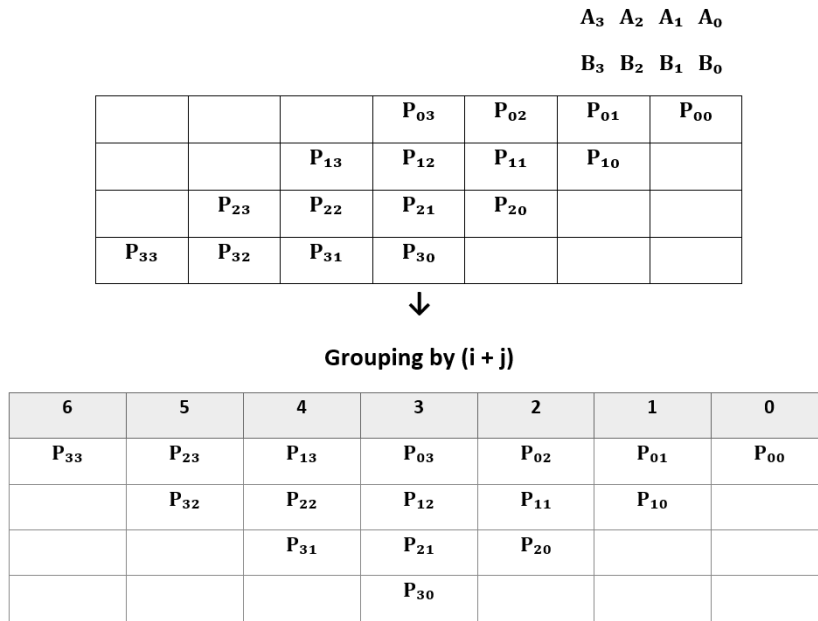


Figure 1: Partial product generation and rearrangement for 4×4 Wallace tree multiplication

The main advantage of the Wallace tree method is that the partial products are reduced in parallel. As illustrated in Fig. 2, whenever three bits appear in the same column, they are compressed using a full adder, and when two bits remain, a half adder may be used if required. In this way, the number of rows is reduced stage by stage until only two rows remain. This reduction process avoids long sequential addition chains and significantly lowers the critical path delay.

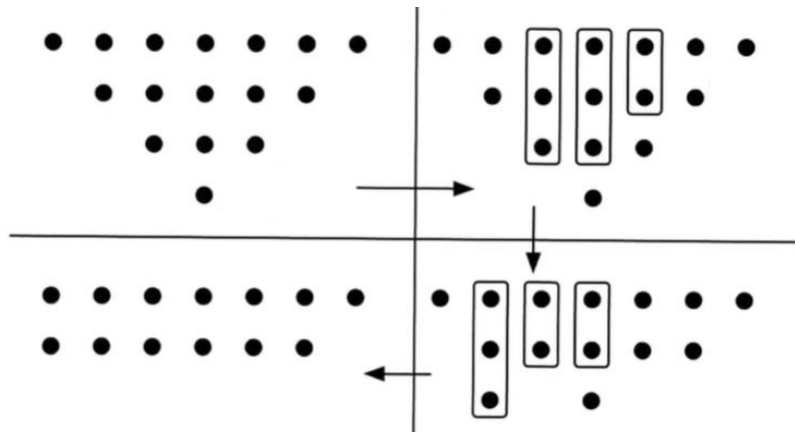


Figure 2: Parallel reduction of partial products in Wallace tree multiplication

The actual compression process used in this project is shown in Fig. 3. The generated partial products are grouped column wise and reduced through successive stages using half adders and full adders. Intermediate sum and carry outputs are formed at each stage. The sum remains in the same column, while the carry is transferred to the next higher weight column. After completing the Wallace reduction, only two final rows remain: one sum row and one carry row. These two rows are then added using a carry propagation adder to produce the final multiplication result.

4 Implementation

4.1 RTL

4.1.1 Code

```
`timescale 1ns/1ps

// -----
// Basic building blocks
// -----

module half_adder(
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);
    assign sum    = a ^ b;
    assign carry  = a & b;
endmodule

module full_adder(
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire carry
);
    assign sum    = a ^ b ^ cin;
    assign carry  = (a & b) | (a & cin) | (b & cin);
endmodule

// -----
```

```

// 4x4 Wallace Tree Multiplier (Pipelined)

// Latency: 2 cycles (input sampled -> output valid after 2 rising
edges)

// Throughput: 1 result per cycle (after pipeline fills)

// -----

module alu(

    input  wire      clk,
    input  wire      rst,    // async active-high reset
    input  wire [3:0] a,
    input  wire [3:0] b,
    output reg  [7:0] p
);

    // =====
    // Stage 0: input registers
    // =====
    reg [3:0] a_r, b_r;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            a_r <= 4'b0;
            b_r <= 4'b0;
        end else begin
            a_r <= a;
            b_r <= b;
        end
    end

    // =====
    // Stage 1: Wallace reduction (combinational) -> register 2 rows
    // =====
    // Partial products (ppij = ai & bj), weight = i+j

```

```

wire pp00 = a_r[0] & b_r[0];

wire pp01 = a_r[0] & b_r[1];

wire pp02 = a_r[0] & b_r[2];

wire pp03 = a_r[0] & b_r[3];


wire pp10 = a_r[1] & b_r[0];

wire pp11 = a_r[1] & b_r[1];

wire pp12 = a_r[1] & b_r[2];

wire pp13 = a_r[1] & b_r[3];


wire pp20 = a_r[2] & b_r[0];

wire pp21 = a_r[2] & b_r[1];

wire pp22 = a_r[2] & b_r[2];

wire pp23 = a_r[2] & b_r[3];


wire pp30 = a_r[3] & b_r[0];

wire pp31 = a_r[3] & b_r[1];

wire pp32 = a_r[3] & b_r[2];

wire pp33 = a_r[3] & b_r[3];


// Wallace-tree signals (chosen to end with two 9-bit rows)

wire s4,  c4;    // w1

wire s1,  c1;    // w2 (first compress)

wire s6,  c6;    // w2 (second compress with carry from w1)

wire s2,  c2;    // w3 (compress pp03,pp12,pp21)

wire s5,  c5;    // w3 (compress pp30,s2,c1), leftover c6 remains as
2nd row bit

wire s3,  c3;    // w4 (compress pp13,pp22,pp31)

wire s9,  c9;    // w4 (compress s3,c2,c5)

wire s7,  c7;    // w5 (compress pp23,pp32,c3)

wire s10, c10;   // w5 (compress s7,c9)

wire s8,  c8;    // w6 (compress pp33,c7)

```

```

wire s11, c11; // w6 (compress s8,c10)

wire s12, c12; // w7 (compress c8,c11) -> c12 becomes bit at weight 8

// Column-by-column compression (Wallace style)

half_adder HA_w1 (.a(pp01), .b(pp10), .sum(s4), .carry(c4));

full_adder FA_w2
(.a(pp02), .b(pp11), .cin(pp20), .sum(s1), .carry(c1));

half_adder HA_w2 (.a(s1), .b(c4), .sum(s6), .carry(c6));

full_adder FA_w3a
(.a(pp03), .b(pp12), .cin(pp21), .sum(s2), .carry(c2));

full_adder FA_w3b
(.a(pp30), .b(s2), .cin(c1), .sum(s5), .carry(c5));

full_adder FA_w4a
(.a(pp13), .b(pp22), .cin(pp31), .sum(s3), .carry(c3));

full_adder FA_w4b
(.a(s3), .b(c2), .cin(c5), .sum(s9), .carry(c9));

full_adder FA_w5a
(.a(pp23), .b(pp32), .cin(c3), .sum(s7), .carry(c7));

half_adder HA_w5 (.a(s7), .b(c9), .sum(s10), .carry(c10));

half_adder HA_w6a (.a(pp33), .b(c7), .sum(s8), .carry(c8));
half_adder HA_w6b (.a(s8), .b(c10), .sum(s11), .carry(c11));

half_adder HA_w7 (.a(c8), .b(c11), .sum(s12), .carry(c12));

// Two rows to be added in the final CPA:

// sum_row has one bit per weight, carry_row holds the "leftover"
bits.

wire [8:0] sum_row_s1;
wire [8:0] carry_row_s1;

```

```

assign sum_row_s1[0] = pp00; // w0

assign sum_row_s1[1] = s4; // w1

assign sum_row_s1[2] = s6; // w2

assign sum_row_s1[3] = s5; // w3

assign sum_row_s1[4] = s9; // w4

assign sum_row_s1[5] = s10; // w5

assign sum_row_s1[6] = s11; // w6

assign sum_row_s1[7] = s12; // w7

assign sum_row_s1[8] = 1'b0; // w8


// leftover bit in weight 3 (from HA_w2 carry):
// keep it as the second row bit at the same weight.
wire [8:0] carry_row_tmp = (9'b1 << 3) & {9{c6}};
// c12 is at weight 8 (from HA_w7 carry)
wire [8:0] carry_row_tmp2 = (9'b1 << 8) & {9{c12}};

assign carry_row_s1 = carry_row_tmp | carry_row_tmp2;


// Register stage-1 rows
reg [8:0] sum_r, carry_r;
always @(posedge clk or posedge rst) begin
    if (rst) begin
        sum_r    <= 9'b0;
        carry_r <= 9'b0;
    end else begin
        sum_r    <= sum_row_s1;
        carry_r <= carry_row_s1;
    end
end

// =====

```

```

// Stage 2: final carry-propagate adder -> output register
// =====
wire [9:0] add_res = sum_r + carry_r; // up to 10 bits

always @(posedge clk or posedge rst) begin
    if (rst) begin
        p <= 8'b0;
    end else begin
        p <= add_res[7:0]; // 4x4 product fits in 8 bits
    end
end

endmodule

```

4.2 Implementation of Algorithm

4.2.1 Building Blocks

```

module half_adder(
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);
    assign sum  = a ^ b;
    assign carry = a & b;
endmodule

module full_adder(
    input  wire a,
    input  wire b,
    input  wire cin,

```

```

    output wire sum,
    output wire carry
);
    assign sum    = a ^ b ^ cin;
    assign carry = (a & b) | (a & cin) | (b & cin);
endmodule

```

The code defines two basic modules: a half adder and a full adder, which are used as fundamental building blocks in the design. The half adder adds two 1-bit inputs, *a* and *b*, and generates a sum using XOR ($a \oplus b$) and a carry using AND ($a \& b$). The full adder extends this by adding three 1-bit inputs, *a*, *b*, and *cin*. The sum is produced by XORing all three inputs, while the carry is generated using the expression $(a \& b) \mid (a \& cin) \mid (b \& cin)$, which becomes high when at least two inputs are high. These modules are used in the multiplier to efficiently combine partial products during the reduction stage.

4.2.2 Input Pipeline

```

always @ (posedge clk or posedge rst)
begin

    if (rst) begin
        a_r <= 4'b0;
        b_r <= 4'b0;
    end
    else begin
        a_r <= a;
        b_r <= b;
    end
end

```

The wallace module defines the top-level structure of the multiplier, where *clk* is the clock signal, *rst* is the asynchronous reset, *a* and *b* are 4-bit inputs, and *p* is the 8-bit output. Inside the module, two registers *a_r* and *b_r* are used to store the input values. An always block is used to update these registers on the positive edge of the clock or reset. When *rst* is high, both registers are cleared to zero. Otherwise, the input values *a* and *b* are stored in *a_r* and *b_r*. This stage acts as the input register stage of the pipeline, ensuring that stable input values are used for further processing in the multiplier.

4.2.3 Wallace Tree Multiplication

```
// Partial products (ppij = ai & bj), weight = i+j

wire pp00 = a_r[0] & b_r[0];
wire pp01 = a_r[0] & b_r[1];
wire pp02 = a_r[0] & b_r[2];
wire pp03 = a_r[0] & b_r[3];

wire pp10 = a_r[1] & b_r[0];
wire pp11 = a_r[1] & b_r[1];
wire pp12 = a_r[1] & b_r[2];
wire pp13 = a_r[1] & b_r[3];

wire pp20 = a_r[2] & b_r[0];
wire pp21 = a_r[2] & b_r[1];
wire pp22 = a_r[2] & b_r[2];
wire pp23 = a_r[2] & b_r[3];

wire pp30 = a_r[3] & b_r[0];
wire pp31 = a_r[3] & b_r[1];
wire pp32 = a_r[3] & b_r[2];
wire pp33 = a_r[3] & b_r[3];
```

This part of the code generates the partial products of the multiplier. In binary multiplication, each bit of one input is multiplied with each bit of the other input using an AND operation. Since the inputs are 4-bit numbers, a total of $4 \times 4 = 16$ partial products are produced. Each partial product is represented as pp_{ij} , where i indicates the bit position of a_r and j indicates the bit position of b_r . For example, $pp_{00} = a_r[0] \& b_r[0]$, $pp_{01} = a_r[0] \& b_r[1]$, and $pp_{23} = a_r[2] \& b_r[3]$. The $weight = i + j$ indicates the bit position where each partial product contributes in the final multiplication result. Thus, this section forms the basic multiplication terms that will later be reduced using the Wallace tree structure.

```

full_adder FA_w2(.a(pp02),.b(pp11),.cin(pp20),.sum(s1),.carry(c1));
half_adder HA_w2(.a(s1),.b(c4),.sum(s6),.carry(c6));
full_adder FA_w3a(.a(pp03),.b(pp12),.cin(pp21),.sum(s2),.carry(c2));

full_adder FA_w3b(.a(pp30),.b(s2),.cin(c1),.sum(s5),.carry(c5));
full_adder FA_w4a(.a(pp13),.b(pp22),.cin(pp31),.sum(s3),.carry(c3));
full_adder FA_w4b(.a(s3),.b(c2),.cin(c5),.sum(s9),.carry(c9));

full_adder FA_w5a(.a(pp23),.b(pp32),.cin(c3),.sum(s7),.carry(c7));
half_adder HA_w5(.a(s7),.b(c9),.sum(s10),.carry(c10));

half_adder HA_w6a(.a(pp33),.b(c7),.sum(s8),.carry(c8));
half_adder HA_w6b(.a(s8),.b(c10),.sum(s11),.carry(c11));

half_adder HA_w7(.a(c8),.b(c11),.sum(s12),.carry(c12));

wire[8:0] sum_row_s1;
wire[8:0] carry_row_s1;
assign sum_row_s1[0] = pp00; // w0
assign sum_row_s1[1] = s4; // w1
assign sum_row_s1[2] = s6; // w2
assign sum_row_s1[3] = s5; // w3
assign sum_row_s1[4] = s9; // w4
assign sum_row_s1[5] = s10; // w5
assign sum_row_s1[6] = s11; // w6
assign sum_row_s1[7] = s12; // w7
assign sum_row_s1[8] = 1'b0; // w8

```

This part of the code performs the Wallace tree reduction using half adders and full adders. The generated partial products are grouped according to their bit positions (weights), and then reduced column by column. In each column, multiple bits are combined using full adders (for three inputs) and half adders (for two inputs). The outputs of these adders are a sum and a carry, where the sum remains in the same column and the carry is passed to the next higher

column. This process continues across all bit positions, gradually reducing the number of partial product bits. As a result, the large set of partial products is compressed into only two rows, represented by intermediate sum and carry signals (s and c).

4.2.4 Two Row Decomposition and CPA

```
wire [8:0] carry_row_tmp = (9'b1 << 3) & {9{c6}};

wire [8:0] carry_row_tmp2 = (9'b1 << 8) & {9{c12}};

assign carry_row_s1 = carry_row_tmp | carry_row_tmp2;

// Register stage-1 rows
reg [8:0] sum_r, carry_r;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        sum_r    <= 9'b0;
        carry_r <= 9'b0;
    end else begin
        sum_r    <= sum_row_s1;
        carry_r <= carry_row_s1;
    end
end

// Stage 2: final carry-propagate adder -> output register

wire [9:0] add_res = sum_r + carry_r; // up to 10 bits

always @(posedge clk or posedge rst) begin
    if (rst) begin
        p <= 8'b0;
    end else begin
        p <= add_res[7:0]; // 4x4 product fits in 8 bits
    end
end
```

```
end
```

This part of the code stores the reduced sum and carry rows into the registers `sum_r` and `carry_r` at the positive edge of the clock. If the reset signal is active, both registers are cleared to zero. Otherwise, the reduced outputs `sum_row_s1` and `carry_row_s1` are loaded into these registers. This forms the next pipeline stage of the multiplier. After that, the final result is obtained by adding `sum_r` and `carry_r`, which produces `add_res`. This addition acts as the final carry propagate addition stage of the Wallace tree multiplier. In the last always block, the output register `p` is updated on the positive edge of the clock. When reset is active, `p` is cleared to zero; otherwise, the lower 8 bits of `add_res` are assigned to `p`. Since the product of two 4-bit numbers fits within 8 bits, these 8 bits represent the final multiplication result.

4.3 Directed Testbench

4.3.1 Code

```
`timescale 1ns/1ps

module tb_alu;

    reg        clk;
    reg        rst;
    reg [3:0]   a;
    reg [3:0]   b;
    wire [7:0]  p;

    // DUT
    alu dut (
        .clk(clk),
        .rst(rst),
        .a  (a),
        .b  (b),
        .p  (p)
    );

    // Clock generation : 10 ns period

    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // Cadence waveform dump for SimVision

    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb_alu);
    end
end
```

```

// Reference pipeline for 2-cycle latency checking

reg [3:0] a_d0, a_d1;
reg [3:0] b_d0, b_d1;
reg [7:0] exp_d0, exp_d1;
reg      vld_d0, vld_d1;
reg      stim_valid;

integer i, j;
integer case_no;
integer pass_count;
integer fail_count;
integer reset_pass;
integer reset_fail;

// Apply a real test vector

task apply_vec;
input [3:0] ta;
input [3:0] tb;
begin
    @(negedge clk);
    a      = ta;
    b      = tb;
    stim_valid = 1'b1;
end
endtask

// Apply dummy vector for pipeline flush only
// Not counted as a functional test case

task apply_dummy;
begin
    @(negedge clk);
    a      = 4'd0;
    b      = 4'd0;
    stim_valid = 1'b0;
end
endtask

// Checker / scoreboard
// Since DUT latency = 2 cycles, output p must match
// exp_d1 when vld_d1 is high

always @(posedge clk or posedge rst) begin
    if (rst) begin
        a_d0 <= 4'd0;
        a_d1 <= 4'd0;
        b_d0 <= 4'd0;
        b_d1 <= 4'd0;
        exp_d0 <= 8'd0;
        exp_d1 <= 8'd0;
        vld_d0 <= 1'b0;
        vld_d1 <= 1'b0;
    end
    else begin
        #1; // let registered DUT outputs settle

        if (vld_d1) begin

```

```

        case_no = case_no + 1;

        if (p === exp_d1) begin
            pass_count = pass_count + 1;
            $display("CASE %0d : a = %0d, b = %0d, y = %0d, expected = %0d
--> PASSED",
                    case_no, a_d1, b_d1, p, exp_d1);
        end
        else begin
            fail_count = fail_count + 1;
            $display("CASE %0d : a = %0d, b = %0d, y = %0d, expected = %0d
--> FAILED",
                    case_no, a_d1, b_d1, p, exp_d1);
        end
    end
end

// shift reference pipeline
a_d1  <= a_d0;
b_d1  <= b_d0;
exp_d1 <= exp_d0;
vld_d1 <= vld_d0;

a_d0  <= a;
b_d0  <= b;
exp_d0 <= a * b;
vld_d0 <= stim_valid;
end
end

// -----
// Main stimulus
// -----
initial begin
    // initial values
    rst      = 1'b1;
    a        = 4'd0;
    b        = 4'd0;
    stim_valid = 1'b0;

    case_no   = 0;
    pass_count = 0;
    fail_count = 0;
    reset_pass = 0;
    reset_fail = 0;

    // -----
    // Initial asynchronous reset check
    // -----
    #2;
    if (p === 8'd0) begin
        reset_pass = reset_pass + 1;
        $display("RESET CHECK 1 : y = %0d, expected = 0 --> PASSED", p);
    end
    else begin
        reset_fail = reset_fail + 1;
        $display("RESET CHECK 1 : y = %0d, expected = 0 --> FAILED", p);
    end
end

@(negedge clk);
rst = 1'b0;

// -----

```

```

// Directed corner cases
// -----
apply_vec(4'd0 , 4'd0 );
apply_vec(4'd0 , 4'd7 );
apply_vec(4'd9 , 4'd0 );
apply_vec(4'd1 , 4'd1 );
apply_vec(4'd1 , 4'd15);
apply_vec(4'd15, 4'd1 );
apply_vec(4'd2 , 4'd8 );
apply_vec(4'd8 , 4'd2 );
apply_vec(4'd3 , 4'd5 );
apply_vec(4'd5 , 4'd3 );
apply_vec(4'd7 , 4'd7 );
apply_vec(4'd15, 4'd15);
apply_vec(4'd15, 4'd14);
apply_vec(4'd14, 4'd15);
apply_vec(4'd10, 4'd5 );
apply_vec(4'd12, 4'd3 );

// flush directed cases
apply_dummy;
apply_dummy;

// -----
// Mid-stream reset check
// -----
apply_vec(4'd9 , 4'd9 );
apply_vec(4'd11, 4'd13);

#3;
rst = 1'b1;
#1;

if (p == 8'd0) begin
    reset_pass = reset_pass + 1;
    $display("RESET CHECK 2 : y = %0d, expected = 0 --> PASSED", p);
end
else begin
    reset_fail = reset_fail + 1;
    $display("RESET CHECK 2 : y = %0d, expected = 0 --> FAILED", p);
end

a      = 4'd0;
b      = 4'd0;
stim_valid = 1'b0;

@(negedge clk);
rst = 1'b0;

// -----
// Exhaustive test: all 256 combinations
// -----
for (i = 0; i < 16; i = i + 1) begin
    for (j = 0; j < 16; j = j + 1) begin
        apply_vec(i[3:0], j[3:0]);
    end
end

// flush last outputs
apply_dummy;
apply_dummy;

```

```

@(posedge clk);
#1;

// -----
// Final summary
// -----
$display(" ");
$display("=====");
$display("FINAL SUMMARY");
$display("=====");
$display("Functional cases PASSED = %0d", pass_count);
$display("Functional cases FAILED = %0d", fail_count);
$display("Total functional cases = %0d", pass_count + fail_count);
$display("Reset checks PASSED = %0d", reset_pass);
$display("Reset checks FAILED = %0d", reset_fail);
$display("Total reset checks = %0d", reset_pass + reset_fail);
$display("=====");

if ((fail_count == 0) && (reset_fail == 0))
    $display("OVERALL RESULT --> ALL TESTS PASSED");
else
    $display("OVERALL RESULT --> SOME TESTS FAILED");

$finish;
end

endmodule

```

4.3.2 Output

Random Input Test:

```

RESET CHECK 1 : y = 0, expected = 0 --> PASSED
CASE 1 : a = 0, b = 0, y = 0, expected = 0 --> PASSED
CASE 2 : a = 0, b = 7, y = 0, expected = 0 --> PASSED
CASE 3 : a = 9, b = 0, y = 0, expected = 0 --> PASSED
CASE 4 : a = 1, b = 1, y = 1, expected = 1 --> PASSED
CASE 5 : a = 1, b = 15, y = 15, expected = 15 --> PASSED
CASE 6 : a = 15, b = 1, y = 15, expected = 15 --> PASSED
CASE 7 : a = 2, b = 8, y = 16, expected = 16 --> PASSED
CASE 8 : a = 8, b = 2, y = 16, expected = 16 --> PASSED
CASE 9 : a = 3, b = 5, y = 15, expected = 15 --> PASSED
CASE 10 : a = 5, b = 3, y = 15, expected = 15 --> PASSED
CASE 11 : a = 7, b = 7, y = 49, expected = 49 --> PASSED
CASE 12 : a = 15, b = 15, y = 225, expected = 225 --> PASSED
CASE 13 : a = 15, b = 14, y = 210, expected = 210 --> PASSED
CASE 14 : a = 14, b = 15, y = 210, expected = 210 --> PASSED
CASE 15 : a = 10, b = 5, y = 50, expected = 50 --> PASSED
CASE 16 : a = 12, b = 3, y = 36, expected = 36 --> PASSED

```

Figure 5: Output of random directed test

Figure shows the simulation result of the proposed pipelined 4×4 Wallace tree multiplier. First, the reset condition is verified, where the output remains zero, confirming correct reset functionality. Then, multiple directed input combinations are applied to test the multiplier under zero, normal, and maximum operand conditions. In every case, the obtained output y matches the expected multiplication result, and all cases are reported as passed. The

successful verification of cases such as $15 \times 15 = 225$, $14 \times 15 = 210$, and $7 \times 7 = 49$ demonstrates that the multiplier performs correctly for both low and high operand values. Hence, the simulation results confirm the functional correctness and reliability of the implemented design.

Exhaustive Test:

```

CASE 233 : a = 13, b = 8, y = 104, expected = 104 --> PASSED
CASE 234 : a = 13, b = 9, y = 117, expected = 117 --> PASSED
CASE 235 : a = 13, b = 10, y = 130, expected = 130 --> PASSED
CASE 236 : a = 13, b = 11, y = 143, expected = 143 --> PASSED
CASE 237 : a = 13, b = 12, y = 156, expected = 156 --> PASSED
CASE 238 : a = 13, b = 13, y = 169, expected = 169 --> PASSED
CASE 239 : a = 13, b = 14, y = 182, expected = 182 --> PASSED
CASE 240 : a = 13, b = 15, y = 195, expected = 195 --> PASSED
CASE 241 : a = 14, b = 0, y = 0, expected = 0 --> PASSED
CASE 242 : a = 14, b = 1, y = 14, expected = 14 --> PASSED
CASE 243 : a = 14, b = 2, y = 28, expected = 28 --> PASSED
CASE 244 : a = 14, b = 3, y = 42, expected = 42 --> PASSED
CASE 245 : a = 14, b = 4, y = 56, expected = 56 --> PASSED
CASE 246 : a = 14, b = 5, y = 70, expected = 70 --> PASSED
CASE 247 : a = 14, b = 6, y = 84, expected = 84 --> PASSED
CASE 248 : a = 14, b = 7, y = 98, expected = 98 --> PASSED
CASE 249 : a = 14, b = 8, y = 112, expected = 112 --> PASSED
CASE 250 : a = 14, b = 9, y = 126, expected = 126 --> PASSED
CASE 251 : a = 14, b = 10, y = 140, expected = 140 --> PASSED
CASE 252 : a = 14, b = 11, y = 154, expected = 154 --> PASSED
CASE 253 : a = 14, b = 12, y = 168, expected = 168 --> PASSED
CASE 254 : a = 14, b = 13, y = 182, expected = 182 --> PASSED
CASE 255 : a = 14, b = 14, y = 196, expected = 196 --> PASSED
CASE 256 : a = 14, b = 15, y = 210, expected = 210 --> PASSED
=====
FINAL SUMMARY
=====
Functional cases PASSED = 272
Functional cases FAILED = 0
Total functional cases = 272
Reset checks PASSED = 2
Reset checks FAILED = 0
Total reset checks = 2
=====
OVERALL RESULT --> ALL TESTS PASSED
Simulation complete via $finish(1) at time 2806 NS + 0

```

Figure 6: Output of directed testbench

Explanation:

The figure shows the exhaustive verification results of the proposed 4×4 pipelined Wallace tree multiplier, where all the input combinations are applied and the corresponding outputs are compared with the expected multiplication results. Each line in the log represents a single test case, indicating the input values a and b , the simulated output y , and the expected result. For example, the entry “CASE 233: $a=13$, $b=8$, $y = 104$, expected = 104 → PASSED” confirms that the multiplier correctly computes $13 \times 8 = 104$. Similar correctness is observed for higher-value cases such as $14 \times 15 = 210$, demonstrating that the design operates accurately across the full input range. All listed cases are marked as PASSED, indicating that the

computed outputs match the expected values in every instance.

The summary section shows that a total of 272 functional test cases were executed, out of which all 272 cases passed and none failed. Additionally, 2 reset checks were performed, both of which also passed successfully. This means that the design achieved 100% success in both functional verification and reset validation. The absence of any failed cases confirms that the multiplier is functionally correct for all tested inputs, and the correct reset behavior ensures reliable initialization.

4.3.3 Simulation Result

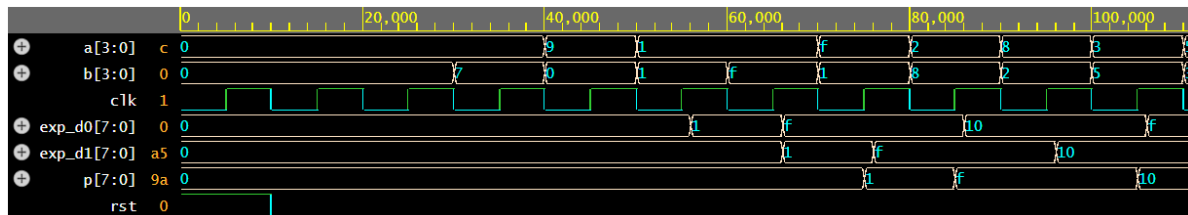


Figure 7: Simulation Output of Directed Testbench

The figure shows the waveform simulation of the pipelined 4×4 Wallace tree multiplier, illustrating the temporal relationship between inputs, clock, and output signals. The input operands a[3:0] and b[3:0] change at different time intervals, while the clock signal (clk) drives the sequential pipeline stages. The reset signal (rst) is initially asserted to initialize the system, forcing the output p[7:0] to zero, and then deasserted to allow normal operation. The expected intermediate outputs (exp_d0 and exp_d1) represent pipeline-aligned reference values used for verification.

Due to the pipelined architecture, the output p[7:0] does not immediately reflect the current inputs but appears after a latency of two clock cycles. This delay corresponds to the two-stage pipeline structure of the design. For example, when specific input values are applied, the corresponding expected outputs (exp_d1) appear first, followed by the actual output p after the required clock cycles, matching exactly with the expected values. The alignment between exp_d1 and p confirms correct pipeline synchronization and data propagation.

Overall, the waveform verifies that the multiplier produces correct results with proper timing behavior. The output consistently matches the expected values after the pipeline delay, confirming both functional correctness and correct implementation of the pipelined Wallace tree architecture.

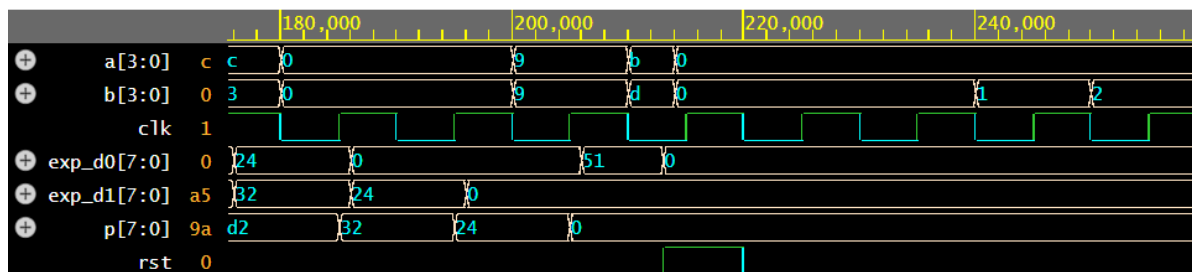


Figure 8: Output of midstream reset

The figure shows the effect of the reset (rst) signal on the pipelined multiplier operation. When the reset is asserted (logic high), all internal pipeline registers are cleared, and the output

p[7:0] is immediately forced to zero regardless of the input values. This ensures that the system starts from a known and stable state. During this period, previously computed or partially propagated values in the pipeline are discarded.

After the reset signal is deasserted (logic low), the multiplier resumes normal operation. New input values are captured at the next clock edge, and valid outputs begin to appear again after the inherent pipeline latency. The waveform confirms that the reset effectively flushes the pipeline and prevents any invalid or residual data from propagating to the output, thereby ensuring correct initialization and reliable operation of the design.

4.4 Layered Testbench

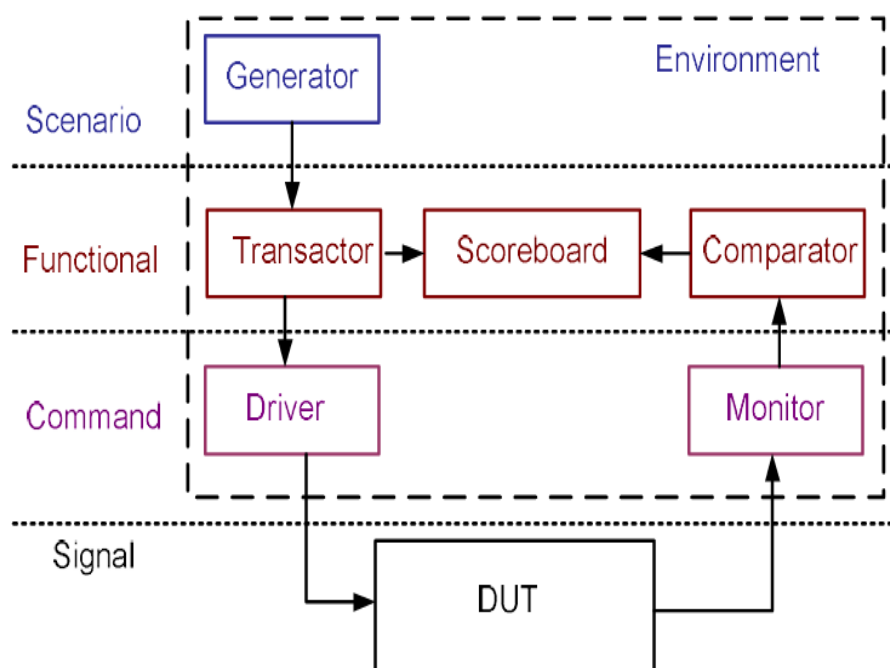


Figure 9: Modules of layered testbench

The layered testbench is organized into several verification components that work together to validate the pipelined Wallace tree multiplier. The generator creates different multiplication scenarios, while the transaction object stores the input operands, expected results, and related verification data in a structured form. The driver applies these transactions to the DUT according to the required clock and reset timing, and the monitor passively captures the output responses. These captured outputs are checked by the comparator against the expected multiplication results, while the scoreboard records pass/fail information and summarizes the overall verification status. All of these components are integrated and managed by the environment, and the DUT represents the 4×4 pipelined Wallace tree multiplier being verified.

4.4.1 Code

driver.sv

```

class driver;

virtual
alu_if
vif;

mailbox # (transaction) gen2drv;
mailbox # (transaction) drv2sb;

int
reset_pass;
int
reset_fail;

function
new(
    virtual
alu_if
vif,
mailbox # (transaction) gen2drv,
mailbox # (transaction) drv2sb
);
this.vif = vif;
this.gen2drv = gen2drv;
this.drv2sb = drv2sb;
this.reset_pass = 0;
this.reset_fail = 0;
endfunction

task
automatic
reset_dut(string
tag = "RESET");
vif.rst = 1
'b1;
vif.a = 4
'd0;
vif.b = 4
'd0;
vif.tb_valid = 1
'b0;

# 1;
if (vif.p === 8'd0) begin
reset_pass++;
$display("[DRV] %s PASSED: p = %0d", tag, vif.p);
end
else begin
reset_fail++;
$display("[DRV] %s FAILED: p = %0d, expected = 0", tag, vif.p);
end

repeat (2) @ (negedge vif.clk);
vif.rst = 1'b0;
$display("[DRV] %s deasserted.", tag);
endtask

task automatic midstream_reset(string tag = "MIDSTREAM RESET");
vif.rst      = 1'b1;
vif.a        = 4'd0;
vif.b        = 4'd0;

```

```

vif.tb_valid = 1'b0;

# 1;
if (vif.p == 8'd0) begin
    reset_pass++;
    $display("[DRV] %s PASSED: p = %0d", tag, vif.p);
end
else begin
    reset_fail++;
    $display("[DRV] %s FAILED: p = %0d, expected = 0", tag, vif.p);
end

@ (negedge vif.clk);
vif.rst = 1'b0;
$display("[DRV] %s deasserted.", tag);
endtask

task automatic drive_one(transaction tr, bit send_to_scoreboard = 1'b1);
@ (negedge vif.clk);
vif.a      = tr.a;
vif.b      = tr.b;
vif.tb_valid = 1'b1;

if (send_to_scoreboard)
    drv2sb.put(tr.copy());
endtask

task automatic drive_n(int total_txns);
transaction tr;

repeat (total_txns) begin
    gen2drv.get(tr);
    drive_one(tr, 1'b1);
end
endtask

task automatic apply_dummy(int n = 1);
repeat (n) begin
    @ (negedge vif.clk);
    vif.a      = 4'd0;
    vif.b      = 4'd0;
    vif.tb_valid = 1'b0;
end
endtask

endclass

```

driver.sv defines the driver class, which is responsible for converting high-level transactions into actual DUT input signals through the virtual interface vif. It receives transactions from the generator through the gen2drv mailbox and forwards copies of the applied transactions to the scoreboard through the drv2sb mailbox for result checking. The constructor initializes the interface, mailbox connections, and reset pass/fail counters. The file also implements both reset_dut() and midstream_reset() to verify correct reset behavior at the beginning of simulation as well as during active pipeline operation. Functional input application is handled by drive_one() and drive_n(), where transactions are applied to the DUT in a clock-synchronized manner, ensuring proper operation of the pipelined multiplier. In

addition, `apply_dummy()` is used to insert idle cycles for pipeline flushing and timing alignment. Overall, this file performs signal-level stimulus application, reset verification, and coordination with the scoreboard for structured functional testing.

```
class generator;

mailbox # (transaction) gen2drv;

function
new(mailbox # (transaction) gen2drv);
this.gen2drv = gen2drv;
endfunction

task
gen_random(int
total_txns);
transaction
tr;

for (int idx = 0; idx < total_txns; idx++)
begin
tr = new();

if (!tr.randomize()) begin
$display("[GEN] ERROR: Randomization failed at idx=%0d", idx);
$finish;
end

tr.id = idx;
gen2drv.put(tr);
end

$display("[GEN] Generated %0d random transactions.", total_txns);
endtask

endclass
```

`generator.sv` defines the generator class, which is responsible for creating the input transactions used during verification. It communicates with the driver through the `gen2drv` mailbox, where each generated transaction is placed for later application to the DUT. The constructor initializes this mailbox connection so that generated test data can be transferred properly. The main operation of this file is performed by the `gen_random()` task, which creates a specified number of transaction objects and randomizes their contents. If randomization fails for any transaction, an error message is displayed and the simulation is stopped immediately to avoid invalid testing. For every successfully generated transaction, a unique transaction ID is assigned and the transaction is sent to the driver through the mailbox. After completing the loop, the generator reports the total number of random transactions created. Overall, this file is responsible for producing randomized functional test cases that help verify the multiplier under a wide range of input conditions.

```
class transaction;
```

```

rand
bit[3:0]
a;
rand
bit[3:0]
b;
bit[7:0]
p;
int
id;

function
new();
a = 4
'd0;
b = 4
'd0;
p = 8
'd0;
id = -1;
endfunction

function
automatic
transaction
copy();
transaction
tr;
tr = new();
tr.a = this.a;
tr.b = this.b;
tr.p = this.p;
tr.id = this.id;
return tr;
endfunction

function
automatic
bit[7:0]
expected_product();
expected_product = {4
'b0, a} * {4'
b0, b};
endfunction

endclass

```

transaction.sv defines the transaction class, which is used as the basic data container for communication between different testbench components. It stores the 4-bit input operands a and b as randomized variables, the 8-bit product p, and an integer id for identifying each transaction uniquely during simulation. The constructor initializes all fields to default values so that every new transaction starts from a known state. The file also implements a copy() function, which creates and returns an exact duplicate of the current transaction object. This is useful when the same transaction data needs to be passed safely between different verification blocks without modifying the original object. In addition, the expected_product() function calculates the reference multiplication result from the input operands by extending them to 8 bits before multiplication. Overall, this file provides the structured transaction

object that carries stimulus, response, and expected result information throughout the layered verification environment.

```
`timescale 1ns/1ps

interface alu_if;

    logic        clk;
    logic        rst;
    logic [3:0]   a;
    logic [3:0]   b;
    logic [7:0]   p;

    // TB-only valid
    logic        tb_valid;

    // Internal DUT observation signals
    logic [3:0]   a_r_obs;
    logic [3:0]   b_r_obs;
    logic [8:0]   sum_r_obs;
    logic [8:0]   carry_r_obs;

endinterface
```

interface.sv defines the `alu_if` interface, which provides a shared connection structure between the DUT and the verification components. It includes the main DUT signals such as clock `clk`, reset `rst`, 4-bit input operands `a` and `b`, and the 8-bit output product `p`. In addition to these design signals, the interface also includes the testbench-only control signal `tb_valid`, which is used to indicate whether the applied input data is valid during simulation. To support internal verification and debugging, the interface further declares observation signals such as `a_r_obs`, `b_r_obs`, `sum_r_obs`, and `carry_r_obs`, which allow the testbench to monitor important internal registered and intermediate DUT values. Overall, this file creates a clean and organized communication layer that simplifies signal access, improves connectivity between testbench blocks and the DUT, and supports both functional verification and internal waveform observation.

```
class testcase03;

    virtual
    alu_if
    vif;
    environment
    env;

    function
    new(virtual
    alu_if
    vif);
    this.vif = vif;
    this.env = new(vif);
    endfunction

    task
    run();
    $display("[TESTCASE03] Starting pipeline / register testcase...");
```

```

env.run_pipeline_test();
$display("[TESTCASE03] Pipeline / register testcase completed.");
endtask

endclass

```

testcase03.sv defines the testcase03 class, which is used to execute a specific verification scenario focused on pipeline and register behavior. It holds a virtual interface handle vif and an instance of the verification environment env. The constructor initializes the interface connection and creates the environment using that interface, so that all verification components can communicate with the DUT properly. The main execution is performed inside the run() task, where a message is displayed at the start and end of the test for simulation traceability. Within this task, the function env.run_pipeline_test() is called to perform the actual pipeline and register verification. Overall, this file acts as a dedicated test case wrapper that triggers a specific verification mode within the layered testbench.

```

class testcase02;

virtual
alu_if
vif;
environment
env;

function
new(virtual
alu_if
vif);
this.vif = vif;
this.env = new(vif);
endfunction

task
run();
$display("[TESTCASE02] Starting reset testcase...");
env.run_reset_test();
$display("[TESTCASE02] Reset testcase completed.");
endtask

endclass

```

testcase02.sv defines the testcase02 class, which is used to execute the reset verification scenario of the DUT. It contains a virtual interface handle vif and an instance of the verification environment env. The constructor initializes the interface connection and creates the environment using that interface, allowing the verification components to interact with the DUT correctly. The main operation is performed inside the run() task, where simulation messages are displayed before and after execution for clear test identification. Inside this task, the method env.run_reset_test() is called to perform the actual reset verification procedure. Overall, this file serves as a dedicated reset test wrapper that activates the corresponding verification flow within the layered testbench.

```
class testcase01;
```

```
virtual  
alu_if  
vif;  
environment  
env;
```

```
function  
new(virtual  
alu_if  
vif);  
this.vif = vif;  
this.env = new(vif);  
endfunction
```

```
task  
run();  
$display("[TESTCASE01] Starting random functional testcase...");  
env.run_random_functional(1024);  
$display("[TESTCASE01] Random functional testcase completed.");  
endtask
```

```
endclass
```

testcase01.sv defines the testcase01 class, which is used to execute the random functional verification scenario of the DUT. It contains a virtual interface handle vif and an instance of the verification environment env. The constructor initializes the interface connection and creates the environment using that interface, so that all verification components can communicate properly with the DUT. The main test execution is performed inside the run() task, where start and completion messages are displayed for clear simulation tracking. Within this task, the method env.run_random_functional(1024) is called to apply 1024 random functional transactions for verification. Overall, this file acts as a dedicated functional test wrapper that triggers randomized testing of the pipelined multiplier within the layered testbench.

```
class monitor;
```

```
virtual  
alu_if  
vif;  
mailbox # (transaction) mon2sb;
```

```
bit  
vld_d0;  
bit  
vld_d1;  
int  
observed_count;
```

```
bit[3:0]  
cov_a;  
bit[3:0]
```

```

cov_b;

covergroup
cg_inputs;
option.per_instance = 1;

cp_a: coverpoint
cov_a
{
    bins
a_vals[] = {[0: 15]};
}

cp_b: coverpoint
cov_b
{
    bins
b_vals[] = {[0: 15]};
}

axb: cross
cp_a, cp_b;
endgroup

function
new(
    virtual
alu_if
vif,
mailbox # (transaction) mon2sb
);
this.vif = vif;
this.mon2sb = mon2sb;
this.vld_d0 = 1
'b0;
this.vld_d1 = 1
'b0;
this.observed_count = 0;
this.cg_inputs = new();
endfunction

task
run();
transaction
tr;

forever
begin

@(posedge vif.clk)

;

if (vif.rst) begin
vld_d0 = 1
'b0;
vld_d1 = 1
'b0;
end
else begin
if (vif.tb_valid) begin

```

```

cov_a = vif.a;
cov_b = vif.b;
cg_inputs.sample();
end

# 1;

if (vld_d1) begin
tr = new();
tr.p = vif.p;
tr.id = observed_count;
mon2sb.put(tr);
observed_count ++;
end

vld_d1 = vld_d0;
vld_d0 = vif.tb_valid;
end
end
endtask

function
real
get_cross_coverage();
return cg_inputs.axb.get_inst_coverage();
endfunction

endclass

```

monitor.sv defines the monitor class, which is responsible for observing DUT activity and capturing output responses for verification. It connects to the DUT through the virtual interface vif and sends observed transactions to the scoreboard using the mon2sb mailbox. The file also includes valid-signal tracking variables vld_d0 and vld_d1, which are used to align output observation with the two-cycle pipeline latency of the multiplier. An observed_count counter is maintained to label the monitored transactions in sequence. In addition, the monitor contains a covergroup cg_inputs that collects functional coverage for all values of input operands a and b, as well as their cross combinations, allowing the verification process to measure how thoroughly the input space has been exercised. Inside the run() task, the monitor continuously samples DUT behavior on each positive clock edge, resets the internal valid-tracking logic during reset, records input coverage whenever tb_valid is asserted, and captures the DUT output p after the required pipeline delay before forwarding it to the scoreboard. The file also provides the get_cross_coverage() function to report the achieved cross-coverage percentage. Overall, this file performs output observation, pipeline-aware response capture, and functional coverage collection within the layered testbench.

```

class environment;

virtual
alu_if
vif;

mailbox # (transaction) gen2drv;

```

```

mailbox # (transaction) drv2sb;
mailbox # (transaction) mon2sb;

generator
gen;
driver
drv;
monitor
mon;
scoreboard
sb;

int
direct_pass;
int
direct_fail;

function
new(virtual
alu_if
vif);
this.vif = vif;

gen2drv = new();
drv2sb = new();
mon2sb = new();

gen = new(gen2drv);
drv = new(vif, gen2drv, drv2sb);
mon = new(vif, mon2sb);
sb = new(drv2sb, mon2sb);

direct_pass = 0;
direct_fail = 0;
endfunction

function
automatic
real
calc_expected_coverage(int
possible_cases, int
total_txns);
real
ratio;
real
miss_prob;
int
k;

ratio = (possible_cases - 1.0) / possible_cases;
miss_prob = 1.0;

for (k = 0; k < total_txns; k = k + 1) begin
    miss_prob = miss_prob * ratio;
end

calc_expected_coverage = (1.0 - miss_prob) * 100.0;
endfunction

task
automatic
check_value(string

```

```

tag, logic[31:0]
got, logic[31:0]
exp);
if (got === exp) begin
direct_pass + +;
$display("[CHK] %s --> PASSED (got=%0d expected=%0d)", tag, got, exp);
end
else begin
direct_fail + +;
$display("[CHK] %s --> FAILED (got=%0d expected=%0d)", tag, got, exp);
end
endtask

// -----
// TESTCASE
01: random
functional + functional
coverage
// -----
task
run_random_functional(int
total_txns = 512);
real
actual_cov;
real
expected_cov;
int
possible_cases;

possible_cases = 256;

fork
mon.run();
join_none

drv.reset_dut("INITIAL RESET");

gen.gen_random(total_txns);
drv.drive_n(total_txns);
drv.apply_dummy(2);

sb.compare_n(total_txns);
sb.report("RANDOM FUNCTIONAL TEST SUMMARY");

actual_cov = mon.get_cross_coverage();
expected_cov = calc_expected_coverage(possible_cases, total_txns);

$display("[ENV] Random tests run           = %0d", total_txns);
$display("[ENV] Possible combinations      = %0d", possible_cases);
$display("[ENV] Actual functional coverage  = %0.2f%%", actual_cov);
$display("[ENV] Expected random coverage   = %0.2f%%", expected_cov);
$display("[ENV] Reset checks passed        = %0d", drv.reset_pass);
$display("[ENV] Reset checks failed        = %0d", drv.reset_fail);

if ((sb.fail_count == 0) && (drv.reset_fail == 0))
$display("[ENV] OVERALL RESULT --> ALL TESTS PASSED");
else
$display("[ENV] OVERALL RESULT --> SOME TESTS FAILED");

disable
fork;
endtask

```

```

// -----
// TESTCASE
02: reset
functionality
test
// -----
task
run_reset_test();
transaction
tr;
transaction
post_q[$];

drv.reset_dut("INITIAL RESET");

// Drive
two
vectors
that
should
be
aborted
by
midstream
reset
tr = new();
tr.a = 4
'd9; tr.b = 4'
d9;
tr.id = 0;
drv.drive_one(tr, 1
'b0);
tr = new();
tr.a = 4
'd11; tr.b = 4'
d13;
tr.id = 1;
drv.drive_one(tr, 1
'b0);

// Assert
reset
before
these
results
can
reach
output
# 3;
drv.midstream_reset("MIDSTREAM RESET");

// Start
monitor
only
after
reset
so
stale
pre - reset
data
does

```

```

not enter
scoreboard
fork
mon.run();
join_none

// Post - reset
directed
vectors
tr = new();
tr.a = 4
'd1; tr.b = 4'
d1;
tr.id = 0;
post_q.push_back(tr);
tr = new();
tr.a = 4
'd2; tr.b = 4'
d8;
tr.id = 1;
post_q.push_back(tr);
tr = new();
tr.a = 4
'd5; tr.b = 4'
d3;
tr.id = 2;
post_q.push_back(tr);
tr = new();
tr.a = 4
'd15; tr.b = 4'
d15;
tr.id = 3;
post_q.push_back(tr);

foreach(post_q[idx])
begin
drv.drive_one(post_q[idx], 1
'b1);
end

drv.apply_dummy(2);

sb.compare_n(post_q.size());
sb.report("RESET TEST SUMMARY");

$display("[ENV] Reset checks passed = %0d", drv.reset_pass);
$display("[ENV] Reset checks failed = %0d", drv.reset_fail);

if ((sb.fail_count == 0) && (drv.reset_fail == 0))
$display("[ENV] OVERALL RESULT --> RESET TEST PASSED");
else
$display("[ENV] OVERALL RESULT --> RESET TEST FAILED");

disable
fork;
endtask

// -----
// TESTCASE
03: pipeline / register
functionality
test

```

```

// -----
task
run_pipeline_test();
transaction
tr;
transaction
seq_q[$];

// -----
    // Part
A: direct
latency / register
check \
// ----- \
    drv.reset_dut("PIPELINE INIT RESET");

tr = new();
tr.a = 4
'd3;
tr.b = 4
'd5;
tr.id = 0;

drv.drive_one(tr, 1
'b0);

// After
1
st
rising
edge: a_r / b_r
must
capture
input, p
still
0
@ (posedge vif.clk); # 1;
check_value("Stage-0 register capture a_r", vif.a_r_obs, 4
'd3);
check_value("Stage-0 register capture b_r", vif.b_r_obs, 4
'd5);
check_value("Output after first edge", vif.p, 8
'd0);

// After
2
nd
rising
edge: output
still
not valid
    @ (posedge vif.clk); # 1;
check_value("Output after second edge", vif.p, 8
'd0);

// After
3
rd
rising
edge: final
output
must

```

```

appear
@ (posedge vif.clk); # 1;
check_value("Output after third edge", vif.p, 8
'd15);

// -----
// Part
B: back - to - back
pipeline
ordering
check \
// ----- \
    drv.reset_dut("PIPELINE RESTART RESET");

fork
mon.run();
join_none

tr = new();
tr.a = 4
'd1; tr.b = 4'
d1;
tr.id = 0;
seq_q.push_back(tr);
tr = new();
tr.a = 4
'd2; tr.b = 4'
d3;
tr.id = 1;
seq_q.push_back(tr);
tr = new();
tr.a = 4
'd4; tr.b = 4'
d5;
tr.id = 2;
seq_q.push_back(tr);
tr = new();
tr.a = 4
'd15; tr.b = 4'
d15;
tr.id = 3;
seq_q.push_back(tr);

foreach(seq_q[idx])
begin
drv.drive_one(seq_q[idx], 1
'b1);
end

drv.apply_dummy(2);

sb.compare_n(seq_q.size());
sb.report("PIPELINE ORDERING TEST SUMMARY");

$display("[ENV] Direct check pass count = %0d", direct_pass);
$display("[ENV] Direct check fail count = %0d", direct_fail);
$display("[ENV] Reset checks passed      = %0d", drv.reset_pass);
$display("[ENV] Reset checks failed      = %0d", drv.reset_fail);

if ((sb.fail_count == 0) && (drv.reset_fail == 0) && (direct_fail ==
0))
$display("[ENV] OVERALL RESULT --> PIPELINE / REGISTER TEST PASSED");

```

```

else
$display("[ENV] OVERALL RESULT --> PIPELINE / REGISTER TEST FAILED");

disable
fork;
endtask

endclass

```

environment.sv defines the environment class, which acts as the top-level controller of the layered verification testbench. It connects all major components including the generator, driver, monitor, and scoreboard using mailboxes (gen2drv, drv2sb, mon2sb) and the virtual interface vif. The constructor initializes all these components and communication channels, ensuring proper coordination between stimulus generation, DUT driving, output monitoring, and result checking. The file implements multiple verification tasks corresponding to different test scenarios. The run_random_functional() task performs randomized functional testing by generating a large number of input combinations, driving them into the DUT, collecting outputs through the monitor, and comparing results using the scoreboard. It also evaluates functional coverage using a probabilistic model and reports both actual and expected coverage. The run_reset_test() task verifies correct reset behavior by applying inputs before and after a midstream reset, ensuring that pre-reset data does not propagate and post-reset operation remains correct. The run_pipeline_test() task checks pipeline behavior in two parts: first validating register capture and latency across clock cycles, and then verifying correct ordering of outputs for back-to-back inputs. Additionally, helper functions such as calc_expected_coverage() estimate theoretical coverage, and check_value() performs direct value comparisons for internal signal validation. The environment also tracks pass/fail counts for direct checks and reset operations and prints a final summary indicating overall test success or failure. Overall, this file coordinates the complete verification flow, manages different test modes, and ensures comprehensive validation of functionality, reset behavior, and pipeline operation of the DUT.

```

class scoreboard;

mailbox # (transaction) drv2sb;
mailbox # (transaction) mon2sb;

int
case_no;
int
pass_count;
int
fail_count;

function
new(
    mailbox # (transaction) drv2sb,
    mailbox # (transaction) mon2sb
);
this.drv2sb = drv2sb;

```

```

this.mon2sb = mon2sb;
this.case_no = 0;
this.pass_count = 0;
this.fail_count = 0;
endfunction

task
automatic
compare_n(int
total_txns);
transaction
exp_tr;
transaction
act_tr;
bit[7:0]
expected;

repeat(total_txns)
begin
drv2sb.get(exp_tr);
mon2sb.get(act_tr);

expected = exp_tr.expected_product();
case_no + +;

if (act_tr.p === expected) begin
pass_count + +;
$display("CASE %0d : a = %0d, b = %0d, y = %0d, expected = %0d -->
PASSED",
        case_no, exp_tr.a, exp_tr.b, act_tr.p, expected);
end
else begin
fail_count + +;
$display("CASE %0d : a = %0d, b = %0d, y = %0d, expected = %0d -->
FAILED",
        case_no, exp_tr.a, exp_tr.b, act_tr.p, expected);
end
end
endtask

function
void
report(string
title = "SCOREBOARD SUMMARY");
$display(" ");
$display("=====");
$display("%s", title);
$display("=====");
$display("Functional cases PASSED = %0d", pass_count);
$display("Functional cases FAILED = %0d", fail_count);
$display("Total functional cases = %0d", pass_count + fail_count);
$display("=====");
endfunction

endclass

```

scoreboard.sv defines the scoreboard class, which is responsible for checking the functional correctness of the DUT output. It receives the expected transaction information from the driver through the drv2sb mailbox and the actual observed DUT response from the monitor through the mon2sb mailbox. The constructor initializes these mailbox

connections and also sets the case counter, pass counter, and fail counter to zero for clean result tracking at the start of simulation. The main checking operation is performed by the `compare_n()` task, where expected and actual transactions are taken in sequence and compared for a specified number of test cases. For each case, the expected multiplication result is calculated using the `expected_product()` function from the transaction object, and this value is compared with the actual DUT output `p`. If both values match, the case is marked as passed; otherwise, it is marked as failed. Each comparison result is displayed with the corresponding input values, output value, and expected value for clear verification logging. The `report()` function then prints a summary of total passed, failed, and executed functional test cases. Overall, this file performs result comparison, maintains pass/fail statistics, and provides the final functional verification summary of the multiplier design.

Testbench.sv

```
`timescale 1ns/1ps

module testbench;

    alu_if vif();

    `ifdef TESTCASE02
        testcase02 tc;
    `elsif TESTCASE03
        testcase03 tc;
    `else
        testcase01 tc;
    `endif

    // DUT
    alu dut (
        .clk (vif.clk),
        .rst (vif.rst),
        .a   (vif.a),
        .b   (vif.b),
        .p   (vif.p)
    );

    // Internal observation mapping
    assign vif.a_r_obs    = dut.a_r;
    assign vif.b_r_obs    = dut.b_r;
    assign vif.sum_r_obs  = dut.sum_r;
    assign vif.carry_r_obs = dut.carry_r;

    // Clock
    initial begin
        vif.clk = 1'b0;
        forever #5 vif.clk = ~vif.clk;
    end

    // Cadence SHM dump
    initial begin
        $shm_open("waves_layered.shm");
        $shm_probe(testbench, "AS");
    end
end
```

```

initial begin
    vif.rst      = 1'b0;
    vif.a        = 4'd0;
    vif.b        = 4'd0;
    vif.tb_valid = 1'b0;

    tc = new(vif);
    tc.run();

    #10;
    $finish;
end

endmodule

```

testbench.sv defines the top-level verification module that connects the DUT with the layered testbench environment. It first creates an instance of the interface `alu_if`, which is used as the common connection point between the DUT and the verification components. Depending on the selected compilation macro, it instantiates one of the available testcases: `testcase01` for random functional verification, `testcase02` for reset verification, or `testcase03` for pipeline and register verification. The DUT is then instantiated and connected to the interface signals for clock, reset, input operands, and output product. In addition, the file maps important internal DUT registers such as `a_r`, `b_r`, `sum_r`, and `carry_r` to interface observation signals, allowing these internal values to be monitored during simulation. A clock generation block is included to produce a periodic clock signal required for synchronous DUT operation. The file also enables waveform dumping using Cadence SHM commands, so that simulation activity can be viewed later in waveform tools. Finally, the initial block initializes all interface signals to default values, creates the selected testcase object, and starts its execution through the `run()` task. After the test completes, the simulation is terminated. Overall, this file acts as the top-level simulation wrapper that instantiates the DUT, selects the required testcase, provides clock and initialization, and launches the complete layered verification flow.

4.4.2 Output

```

Total functional cases = 1024
=====
[ENV] Random tests run           = 1024
[ENV] Possible combinations      = 256
[ENV] Actual functional coverage = 98.83%
[ENV] Expected random coverage  = 98.18%
[ENV] Reset checks passed       = 1
[ENV] Reset checks failed       = 0
[ENV] OVERALL RESULT --> ALL TESTS PASSED
[TESTCASE01] Random functional testcase completed.
Simulation complete via $finish(1) at time 10296 NS + 0
./testbench.sv:52    $finish;
ncsim> exit

coverage setup:
  workdir  : ./cov_work
  dutinst  : testbench(testbench)
  scope    : scope
  testname : test

coverage files:
  model(design data) : ./cov_work/scope/icc_518b47d0_779cbafb.ucm
  data               : ./cov_work/scope/test/icc_518b47d0_779cbafb.ucd

```

Figure 10: Output of testcase01

Testcase 01 performs randomized functional verification by applying 1024 random input combinations to the multiplier and checking the corresponding outputs. Since the multiplier is a 4-bit $\times$ 4-bit design, each input can take 16 possible values, giving a total of

$$16 \times 16 = 256$$

unique input combinations. However, instead of exhaustively testing all combinations one by one, the test uses random stimulus to approximate full coverage.

The theoretical expected coverage for random testing is calculated using the probability model

$$\text{Coverage} = 1 - \left(\frac{N - 1}{N} \right)^k$$

where ($N = 256$) is the total number of unique input combinations and ($k = 1024$) is the number of random tests applied. Substituting the values, the expected coverage becomes approximately

$$1 - \left(\frac{255}{256} \right)^{1024} \approx 98.18\%$$

From the simulation output, the actual observed functional coverage is reported as 98.83%, which is very close to the theoretical expectation. This slight difference occurs due to randomness—some combinations may be hit more frequently while others may be skipped, but overall the distribution remains statistically consistent with the expected model. Additionally, the reset verification shows that one reset check was performed and passed successfully, with no failures. Since both functional correctness and reset behavior are

validated without any mismatch, the environment concludes with the message “ALL TESTS PASSED,” confirming that the design is operating correctly under random stimulus conditions.

```
[TESTCASE02] Starting reset testcase...
[DRV] INITIAL RESET PASSED: p = 0
[DRV] INITIAL RESET deasserted.
[DRV] MIDSTREAM RESET PASSED: p = 0
[DRV] MIDSTREAM RESET deasserted.
CASE 1 : a = 1, b = 1, y = 1, expected = 1 --> PASSED
CASE 2 : a = 2, b = 8, y = 16, expected = 16 --> PASSED
CASE 3 : a = 5, b = 3, y = 15, expected = 15 --> PASSED
CASE 4 : a = 15, b = 15, y = 225, expected = 225 --> PASSED

=====
RESET TEST SUMMARY
=====
Functional cases PASSED = 4
Functional cases FAILED = 0
Total functional cases = 4
=====
[ENV] Reset checks passed = 2
[ENV] Reset checks failed = 0
[ENV] OVERALL RESULT --> RESET TEST PASSED
[TESTCASE02] Reset testcase completed.
Simulation complete via $finish(1) at time 126 NS + 0
```

Figure 11: Output of testcase02

Testcase 02 verifies reset operation in two conditions: at the beginning of simulation and during active operation. In the initial reset, the reset signal is asserted before normal data processing starts, and the output *p* becomes 0, confirming proper initialization of the DUT. After this check, reset is deasserted and normal operation begins. In the midstream reset, reset is asserted while the pipeline is active. This checks whether the internal pipeline registers and intermediate stored values are properly cleared during operation. The output again returns to 0, showing that the pipeline is successfully flushed. After reset is released, fresh input cases are applied, and all four post-reset functional cases pass correctly.

```
[TESTCASE03] Starting pipeline / register testcase...
[DRV] PIPELINE INIT RESET PASSED: p = 0
[DRV] PIPELINE INIT RESET deasserted.
[CHK] Stage-0 register capture a_r --> PASSED (got=3 expected=3)
[CHK] Stage-0 register capture b_r --> PASSED (got=5 expected=5)
[CHK] Output after first edge --> PASSED (got=0 expected=0)
[CHK] Output after second edge --> PASSED (got=0 expected=0)
[CHK] Output after third edge --> PASSED (got=15 expected=15)
[DRV] PIPELINE RESTART RESET PASSED: p = 0
[DRV] PIPELINE RESTART RESET deasserted.
CASE 1 : a = 1, b = 1, y = 1, expected = 1 --> PASSED
CASE 2 : a = 2, b = 3, y = 6, expected = 6 --> PASSED
CASE 3 : a = 4, b = 5, y = 20, expected = 20 --> PASSED
CASE 4 : a = 15, b = 15, y = 225, expected = 225 --> PASSED

=====
PIPELINE ORDERING TEST SUMMARY
=====
Functional cases PASSED = 4
Functional cases FAILED = 0
Total functional cases = 4
=====
[ENV] Direct check pass count = 5
[ENV] Direct check fail count = 0
[ENV] Reset checks passed = 2
[ENV] Reset checks failed = 0
[ENV] OVERALL RESULT --> PIPELINE / REGISTER TEST PASSED
[TESTCASE03] Pipeline / register testcase completed.
Simulation complete via $finish(1) at time 146 NS + 0
./testbench.sv:52 $finish;
ncsim> exit
```

Figure 12: Output of Testcase 03

Testcase 03 verifies the pipeline and register behavior of the multiplier in three aspects. First, it checks internal register capture, confirming that the stage-0 registers correctly store the applied inputs after the first clock edge. Second, it verifies output latency, where the product remains invalid during the first two clock edges and appears correctly at the third edge, matching the expected pipelined behavior. Third, it checks the ordering of back-to-back inputs by applying four consecutive input pairs and confirming that all corresponding outputs appear in the correct sequence without mismatch.

The simulation results show that all direct pipeline checks passed, all four functional cases passed, and no failure was recorded. In addition, both reset checks were also passed successfully. Therefore, the final summary reports PIPELINE / REGISTER TEST PASSED, confirming correct register updating, proper pipeline delay, and accurate ordered processing of consecutive inputs.

4.5 Synthesis

For the synthesis of the verified RTL design, two main files are required: an SDC file and a TCL file. The SDC file defines the design constraints, while the TCL file controls the synthesis flow by applying those constraints and generating the required synthesis reports. Based on these files, synthesis is performed under specified operating conditions to evaluate the timing and power behavior of the design.

In this project, a parameter sweep was carried out during synthesis to observe how different constraints affect the performance of the multiplier. The parameters varied during this process were:

- Clock period
- Synthesis effort
- Setup time
- Hold time
- Input delay
- Output delay

By comparing the synthesis results for different values of these parameters, the design was analyzed and optimized with respect to timing and power performance. This helped in selecting suitable constraint values for achieving a balanced and efficient implementation of the proposed multiplier.

```

1 # setting up time units
2
3 set_units -time lns -capacitance pF
4
5 # setting the clock period 10ns, as period = 1/freq, here, freq = 100MHz
6 set_clock_period 10;
7
8 set_top_module "alu"
9
10 set_clock_port {clk};
11
12 set_reset_port {rst};
13
14 # setting the input ports in a list to a variable
15 set_input_ports {a,b} ;
16
17 # setting the output ports in a list to a variable
18 set_output_ports {p} ;
19
20 # define the clocks
21 create_clock -period ${clock_period} -waveform {0 6} -name func_clk
22 [get_ports ${clock_port}]
23
24 # setting up constraints for the reset signal
25 set_multicycle_path -setup 3 -from [get_ports ${reset_port}]
26 set_multicycle_path -hold 2 -from [get_ports ${reset_port}]
27
28 # Define input delays
29 set_input_delay 0.4 -clock [get_clocks {func_clk}] ${input_ports}
30
31 # Define output delays
32 set_output_delay 0.6 -clock [get_clocks {func_clk}] ${output_ports}

```

Figure 13: SDC file used for defining synthesis constraints of the design

This figure shows the Synopsys Design Constraints (SDC) file where timing parameters such as clock definition, input output delays, and multicycle paths are specified. These constraints guide the synthesis tool to optimize the design for timing and performance.

```

1 #Run Genus in Legacy UI if Genus is invoked with Common UI
2 ::legacy::set_attribute common ui false ;
3 if {[file exists /proc/cpuinfo]} {
4 sh grep "model name" /proc/cpuinfo
5 sh grep "cpu MHz" /proc/cpuinfo
6 }
7 puts "Hostname : [info hostname]"
8 #####
9 ## Preset global variables and attributes
10 #####
11 set DESIGN alu
12 set SYN_EFF medium
13 set MAP_EFF medium
14 set OPT_EFF medium
15 # Directory of PDK
16 #set_pdk_dir /home/wsadiq/buet_flow/GPDK045
17 set_pdk_dir /home/cad/VLSI2Lab/Digital/Library/
18 set_attribute init_lib_search_path $pdk_dir/gsclib045/timing
19 set_attribute init_hdl_search_path /home/wsadiq/buet_flow/rtl
20 set_attribute init_lib_search_path $pdk_dir
21
22 set_attribute init_hdl_search_path ../../rtl
23 ##Set synthesizing effort for each synthesis stage
24 set_attribute syn_generic_effort $SYN_EFF
25 set_attribute syn_map_effort $MAP_EFF
26 set_attribute syn_opt_effort $OPT_EFF
27 #set_attribute library "\
28 slow_vddl0_basicCells.hvt.lib \
29 slow_vddl0_basicCells.lib \
30 slow_vddl0_basicCells.lvt.lib"
31 set_attribute library "\
32 slow_vddl0_basicCells.lib"
33 set_dont_use [get_lib_cells CLK*]
34 set_dont_use [get_lib_cells SDOFF*]
35 set_dont_use [get_lib_cells DLY*]
36 set_dont_use [get_lib_cells HOLD*]
37 # If you dont want to use LVT uncomment this line
38 #set_dont_use [get_lib_cells *LVT*]
39

```

```

38 #set_dont_use [get_lib_cells *LVT*]
39
40 #####
41 ## Load Design
42 #####
43 ##source verillog_files.tcl
44 read_hdl "\
45 ${DESIGN}.v"
46 elaborate ${DESIGN}
47 puts "Runtime & Memory after 'read_hdl'"
48 time_info Elaboration
49 check_design -unresolved
50 #####
51 ## Constraints Setup
52 #####
53 read_sdc alu.sdc
54
55 report timing -encounter >> reports/${DESIGN}_prelim.rpt
56
57
58 #####
59 ## Synthesizing to generic
60 #####
61 syn_generic
62 puts "Runtime & Memory after 'syn_generic'"
63 time_info GENERIC
64 report datapath > reports/${DESIGN}_datapath_generic.rpt
65 generate_reports -outdir reports -tag generic
66 write_db -to_file ${DESIGN}_generic.db
67 report timing -encounter >> reports/${DESIGN}_generic.rpt
68
69
70 ##This synthesizes your code
71 synthesize -to_mapped
72
73 ## This writes all your files
74 write -mapped > alu_synth.v
75
76 ## THESE FILES ARE NOT REQUIRED, THE SDC FILE IS A TIMING FILE
77 write_script > script
78

```

Figure 14: TCL script used for controlling synthesis flow in Cadence Genus

This figure shows the TCL script where synthesis parameters such as design name, library paths, and synthesis effort levels are defined. It also specifies the technology libraries and constraints used by the synthesis tool to generate an optimized gate-level implementation.

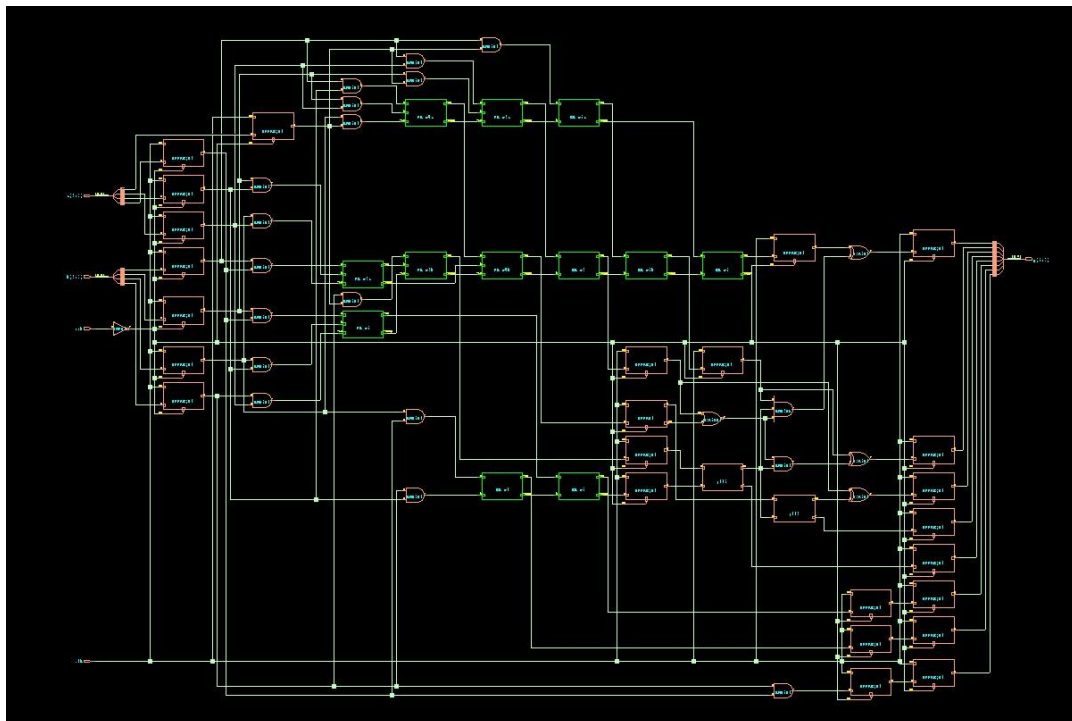


Figure 15: Synthesized gate-level schematic of the 4×4 Wallace tree multiplier

This figure shows the final synthesized circuit generated from the RTL design using Cadence Genus. It represents the optimized gate level implementation, including combinational logic, pipeline registers, and interconnections, reflecting the actual hardware structure after synthesis. The rectangular blocks visible in the schematic mainly correspond to the half adders and full adders used in the Wallace compression and final summation stages.

From the synthesis results(Fig x to Fig y) , it is observed that changing the synthesis effort from medium to high does not produce any noticeable change in either internal power or switching power, while the low-effort option could not be evaluated because it was not supported by the synthesis flow and generated an error. The analysis with varying clock period shows that power consumption decreases significantly as the clock period increases, but this improvement comes at the cost of reduced operating speed. Among the tested values, 10 ns provides the best trade-off between lower power consumption and acceptable performance, and it also gives the lowest leakage power, making it the most suitable clock period for the design. For larger clock periods beyond 10 ns, the additional reduction in power is very small and therefore not worthwhile. Furthermore, the results show that varying input delay, output delay, setup time, and hold time does not cause any meaningful change in power consumption. Overall, the synthesis analysis indicates that the design is largely insensitive to most of the swept timing constraints, and that clock period is the main parameter affecting power-performance trade-off, with 10 ns selected as the optimized operating point.

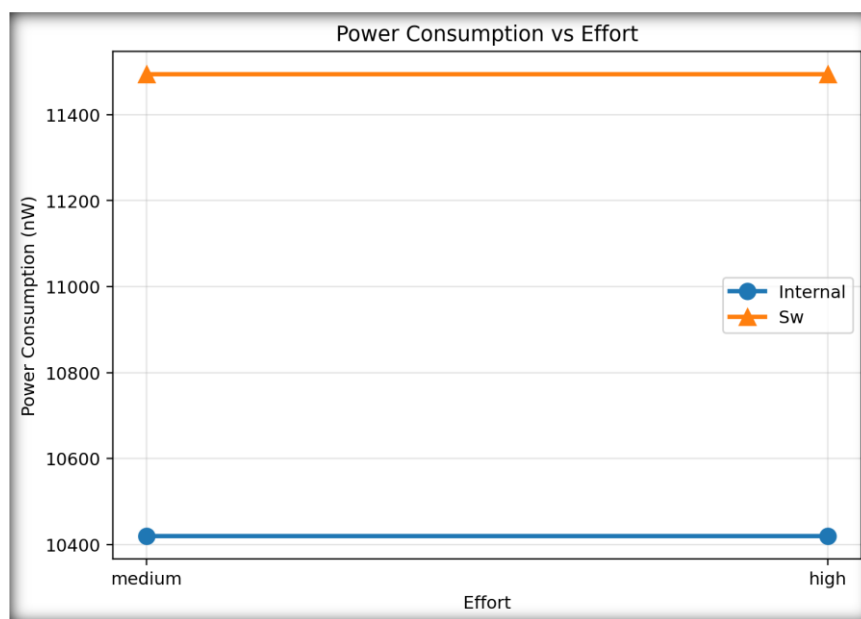


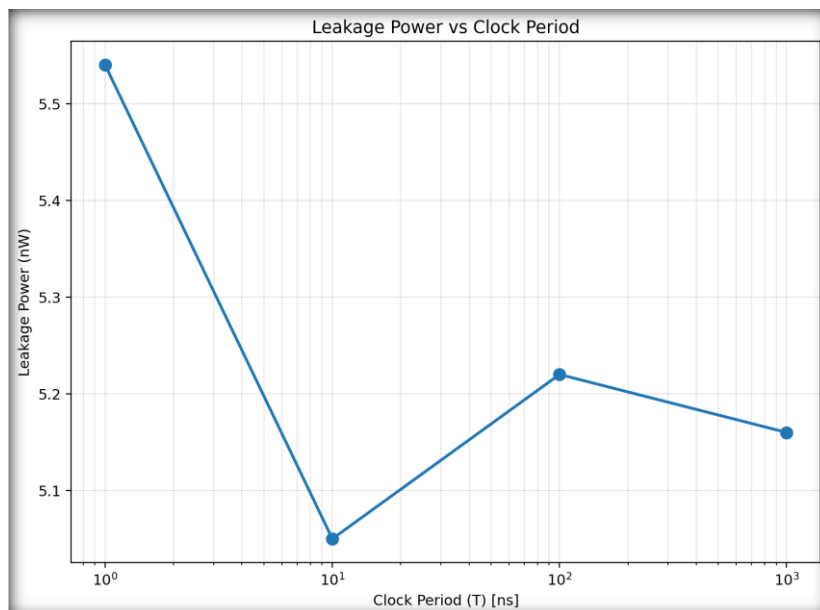
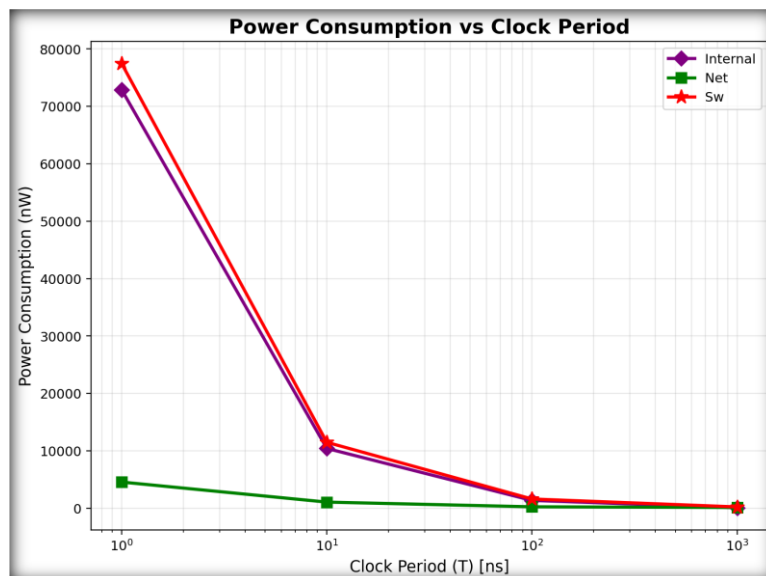
Figure 16 : Power vs Effort

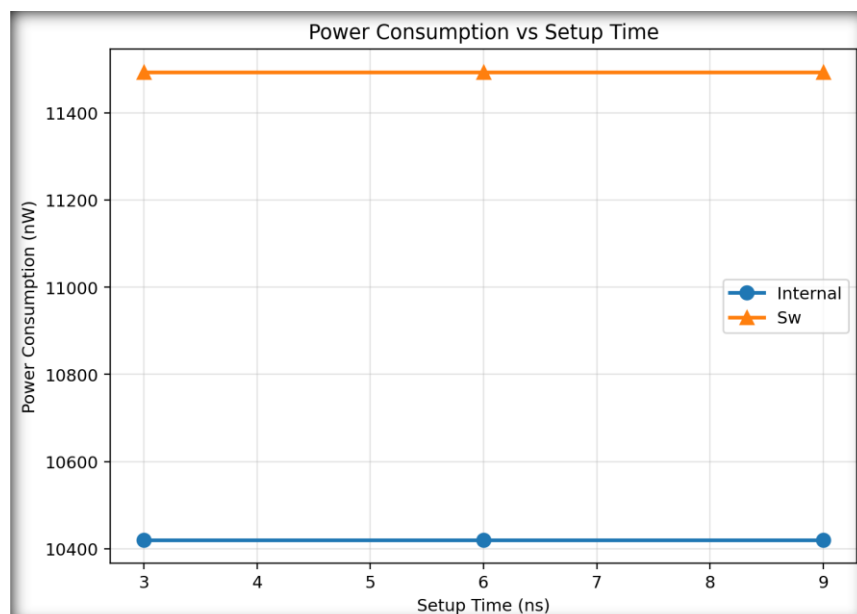
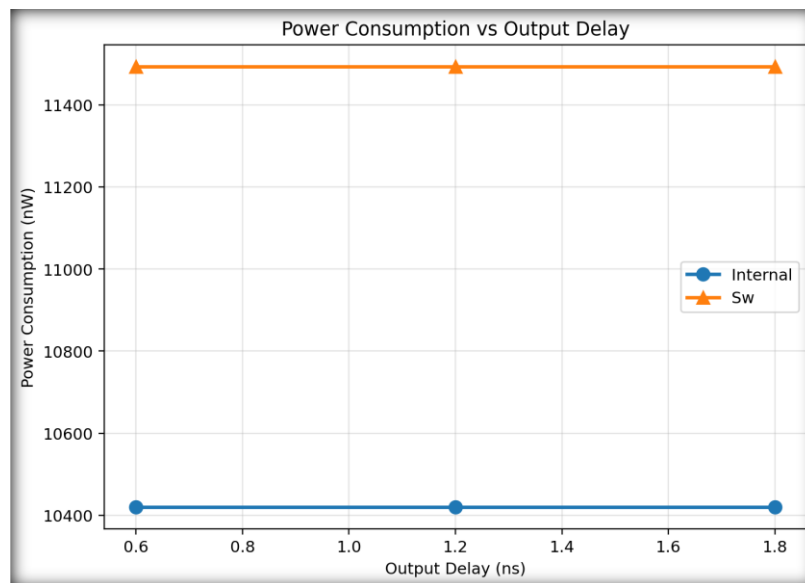
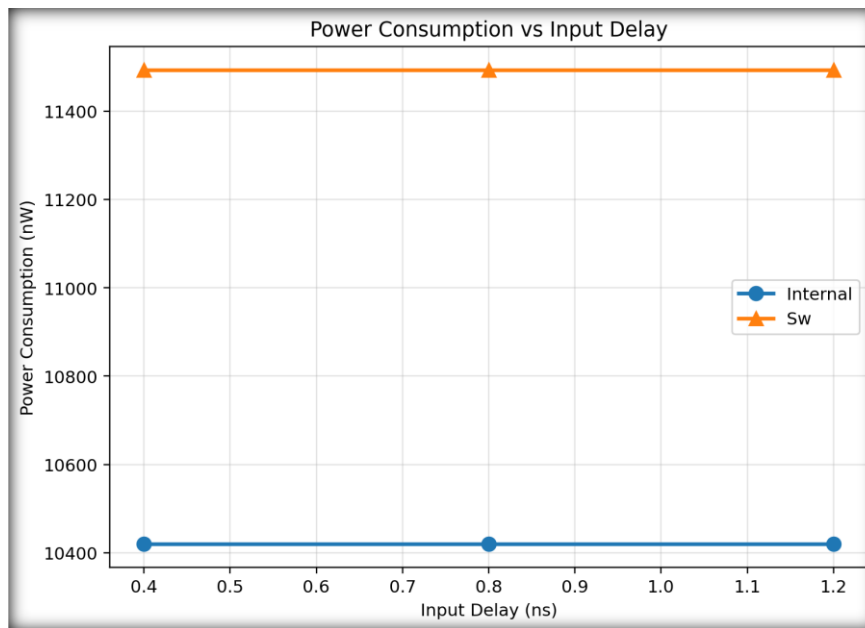
```

"get_ports"          - successful    3 , failed    0 (runtime 0.00)
"set_input_delay"    - successful    1 , failed    0 (runtime 0.00)
"set_multicycle_path" - successful    2 , failed    0 (runtime 0.00)
"set_output_delay"   - successful    1 , failed    0 (runtime 0.00)
"set_units"          - successful    1 , failed    0 (runtime 0.00)
Total runtime 0
Warning : Possible timing problems have been detected in this design. [TIM-11]
: The design is 'alu'.
: Use 'report timing -lint' for more information.
Error : Invalid 'effort'. [SYNTH-24] [syn_generic]
: 'low' value for 'effort' unrecognized for partition based synthesis flow.
: Allowed values for 'effort' are 'medium' and 'high'.
Info : Replacing a flip-flop with a logic constant 0. [GLO-12]
: The instance is 'sum_r_reg[8]'.
: To prevent this optimization, set the 'optimize_constant_0_flops' root att
Info : Replacing a flip-flop with a logic constant 0. [GLO-12]
: The instance is 'carry_r_reg[0]'.
Info : Replacing a flip-flop with a logic constant 0. [GLO-12]
: The instance is 'carry_r_reg[1]'.
Info : Replacing a flip-flop with a logic constant 0. [GLO-12]
: The instance is 'carry_r_reg[2]'.
Info : Replacing a flip-flop with a logic constant 0. [GLO-12]
: The instance is 'carry_r_reg[4]'.
Info : Replacing a flip-flop with a logic constant 0. [GLO-12]
: The instance is 'carry_r_reg[5]'.
Info : Replacing a flip-flop with a logic constant 0. [GLO-12]

```

Figure 17: Low effort value not found error





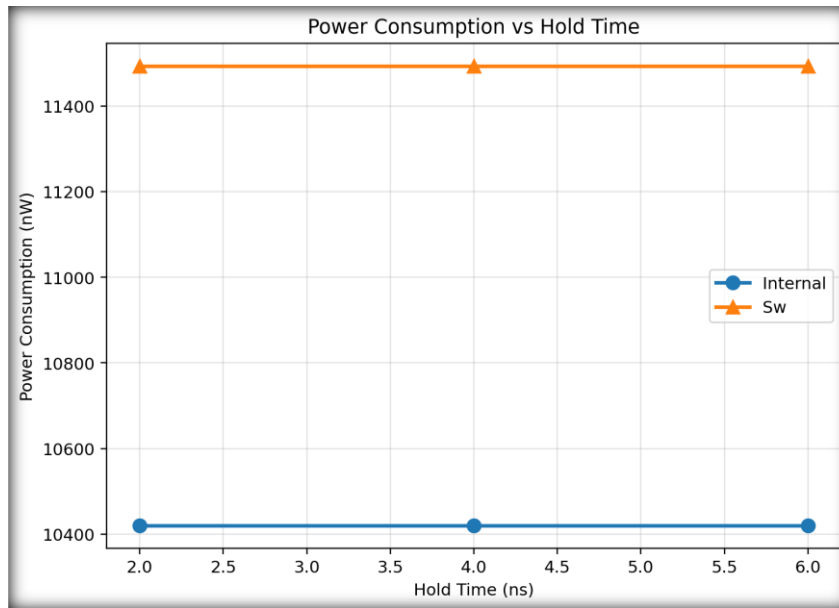


Figure 18: Power vs different parameters

The circuit is mainly composed of half adders and full adders, leaving very limited scope for further optimization. As a result, even when the synthesis effort is increased to high, the power consumption and other performance parameters remain almost unchanged. The final optimized values selected for these parameters are presented in the table below.

Parameter Name	Value
Input Delay	0.4ns
Output Delay	0.6ns
Setup Time	3ns
Hold Time	2ns
Clock Period	10ns
Effort	Medium

4.6 Physical Design

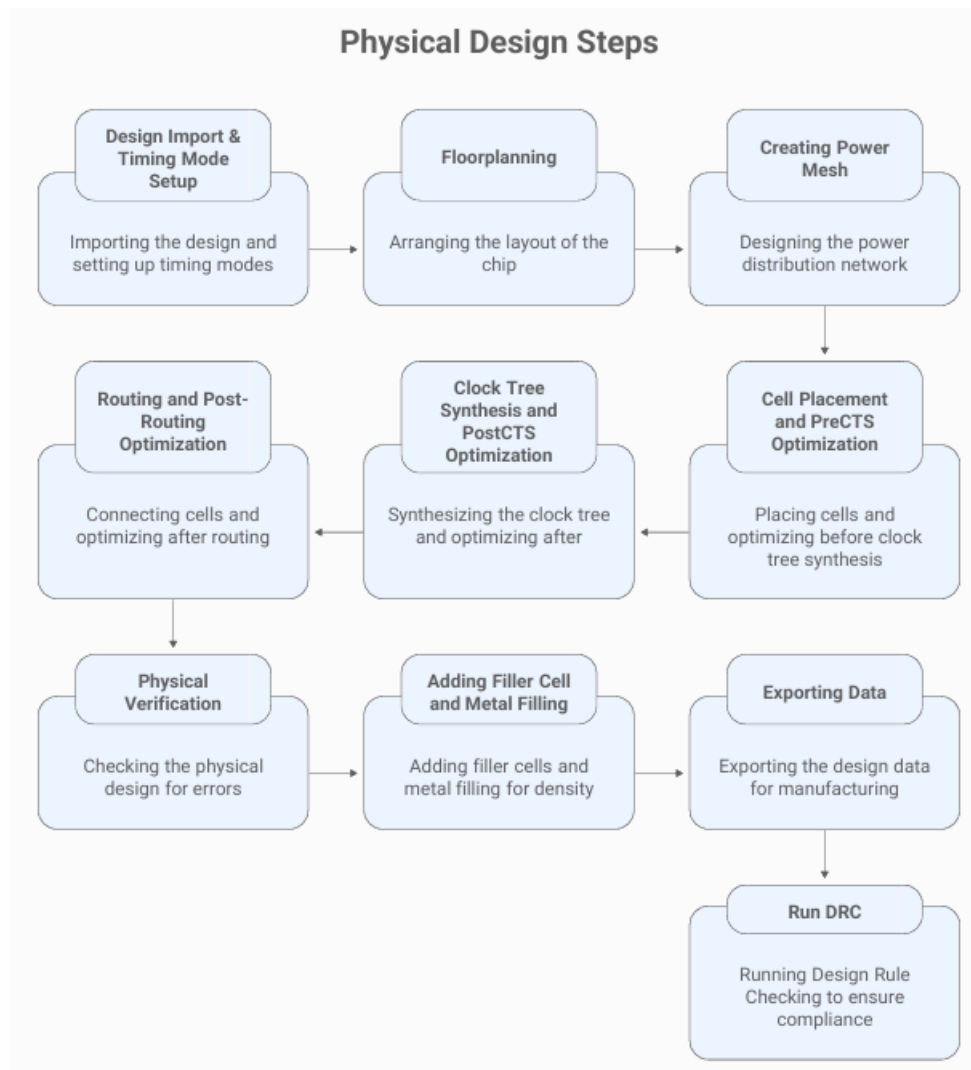


Figure 19: Steps of Physical Design

Physical design is the sequence of backend implementation steps used to convert the synthesized netlist into a layout ready for tape out. In this project, the flow begins with design import and timing-mode setup, followed by floorplanning and IO pin placement to define the physical structure of the design. A power mesh is then created for reliable supply distribution, after which standard cells are placed and optimized before clock tree synthesis. Next, clock tree synthesis, routing, and post route optimization are performed to achieve proper timing and connectivity. Finally, physical verification, filler and metal fill insertion, data export, and DRC checking are carried out to ensure that the layout is physically correct, manufacturable, and ready for final signoff.

4.6.1 Design Import & Timing Mode Setup

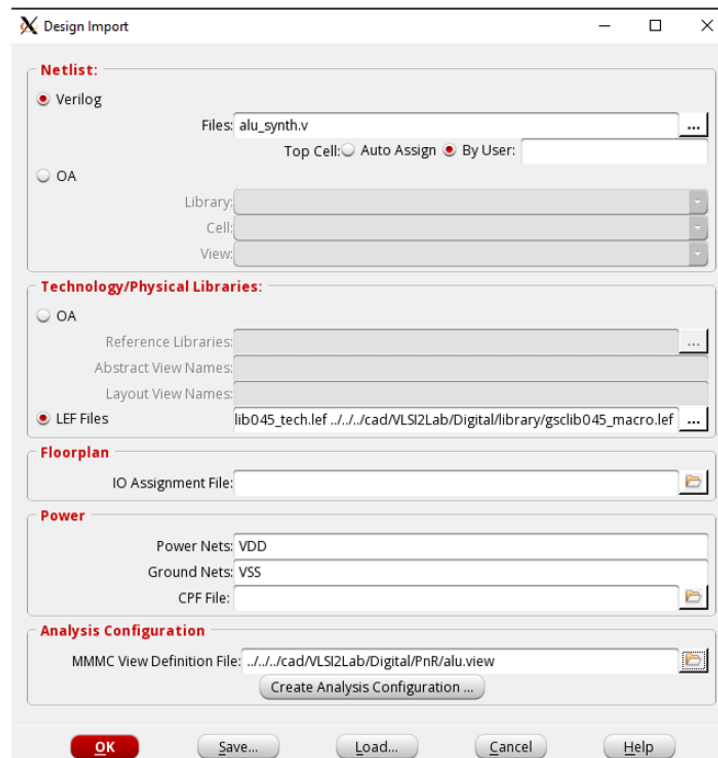


Figure 20: Design Import and Timing Setup in Innovus

The design import configuration window used to initialize the physical design flow shows all the parameters set for design. The synthesized Verilog netlist ('alu_synth.v') is loaded as the top module, and the required technology and standard-cell LEF files are included to define the physical layout rules. Power and ground nets are assigned as VDD and VSS, respectively. Additionally, the MMMC view file is specified to enable multi-mode, multi-corner timing analysis. This step establishes the link between logical design, physical libraries, and timing constraints, forming the foundation for subsequent physical implementation.

4.6.2 Floorplanning

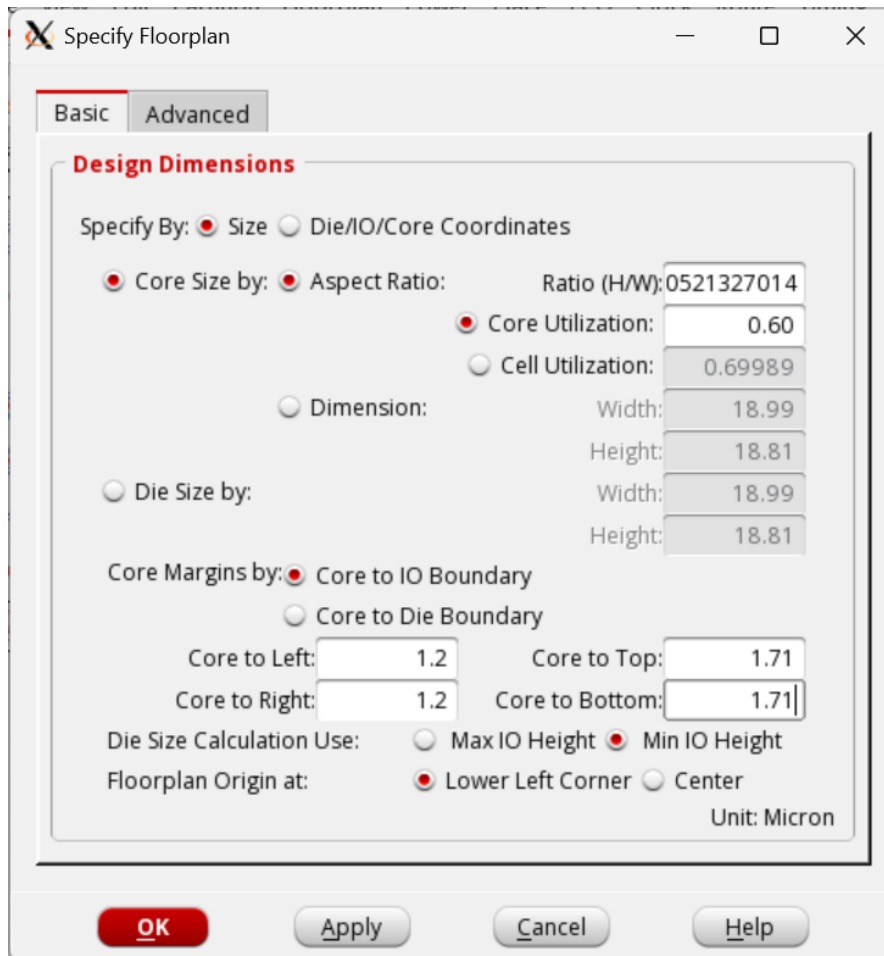


Figure21 : Floorplan Specification in Innovus

The core layout structure is defined by selecting size-based floorplanning and setting the core using aspect ratio and utilization. A core utilization of 0.60 is chosen to keep enough free area for routing, clock distribution, and optimization, while still maintaining reasonable area efficiency. The aspect ratio determines the shape of the core so that placement remains balanced and congestion can be reduced. Instead of forcing exact width and height, these values are automatically derived from the utilization setting and design requirements.

The boundary settings are chosen using Core to IO Boundary, which reserves dedicated space between the core and the IO region. The left and right margins are kept at 1.2 μm while the top and bottom margins are set to 1.71 μm , providing adequate clearance for pin access, routing resources, and power planning around the core. The die size calculation is based on minimum IO height to keep the layout compact, and the floorplan origin is set at the lower-left corner for standard coordinate referencing.

4.6.3 Creating Power Mesh

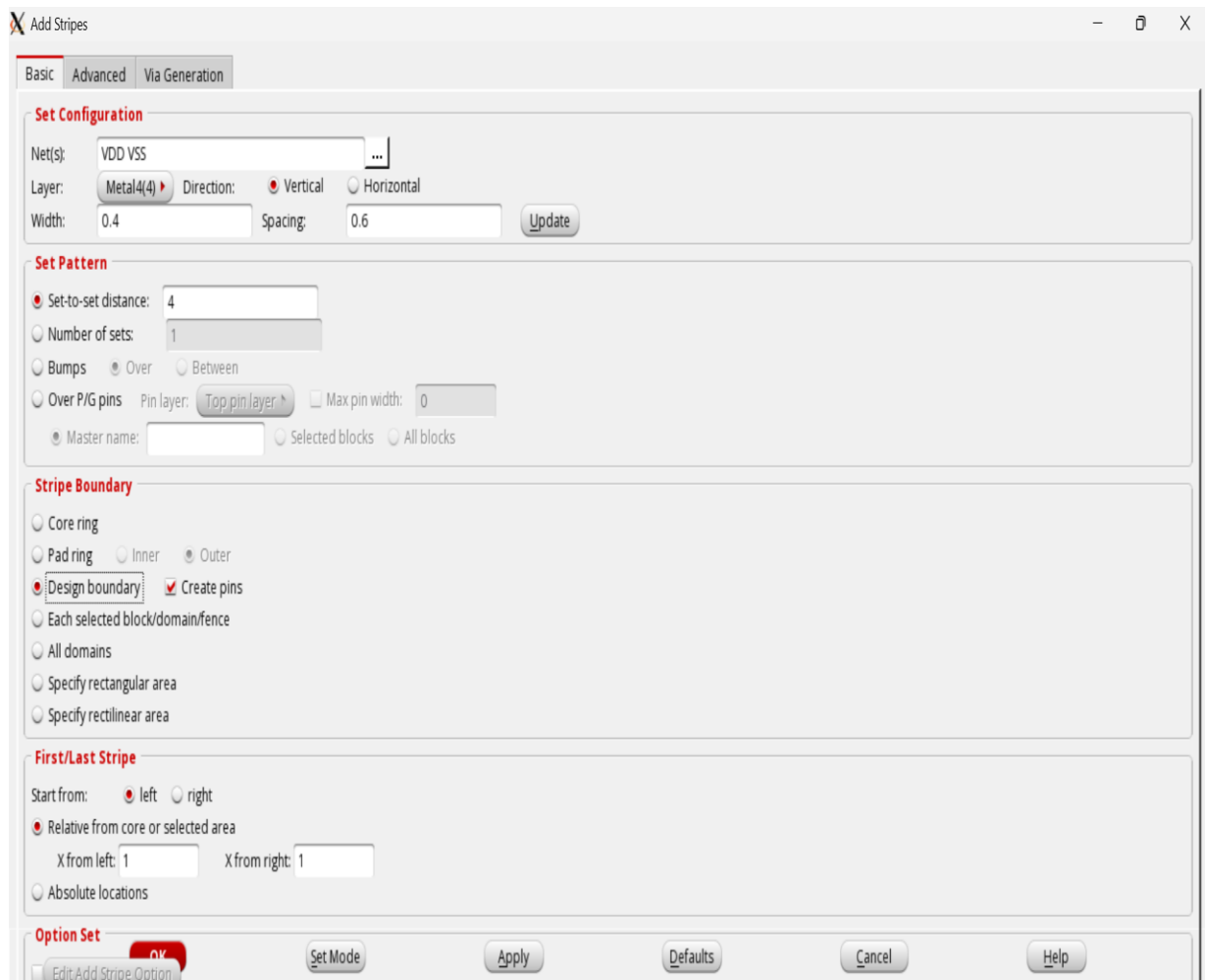


Figure 22: Power Stripe Configuration in Innovus

The power grid is defined by selecting Metal4 as the routing layer and assigning both VDD and VSS nets to ensure proper supply distribution across the core. A stripe width of $0.4\ \mu\text{m}$ is chosen to provide sufficient current carrying capability while avoiding excessive metal usage that could increase congestion. The spacing of $0.6\ \mu\text{m}$ is maintained to ensure design rule compliance and to reduce coupling between adjacent stripes.

The set-to-set distance of $4\ \mu\text{m}$ determines how frequently the power stripe pairs are inserted across the core, which helps maintain uniform voltage distribution and reduces IR drop. Vertical direction is selected because it complements the horizontal routing of lower metal layers, forming an orthogonal power grid structure. The design boundary option ensures that stripes cover the entire core area, while enabling pin creation allows automatic connectivity to power pins, simplifying later routing stages.

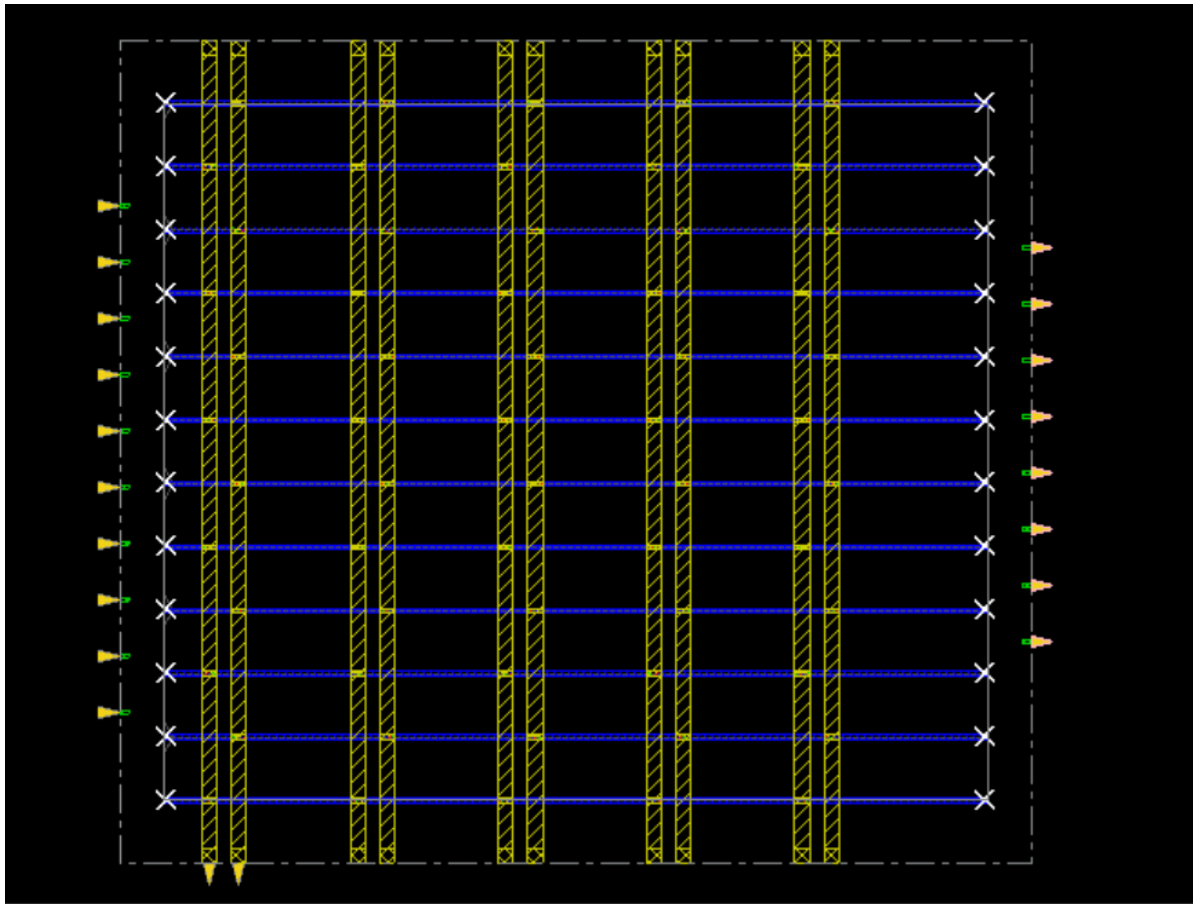
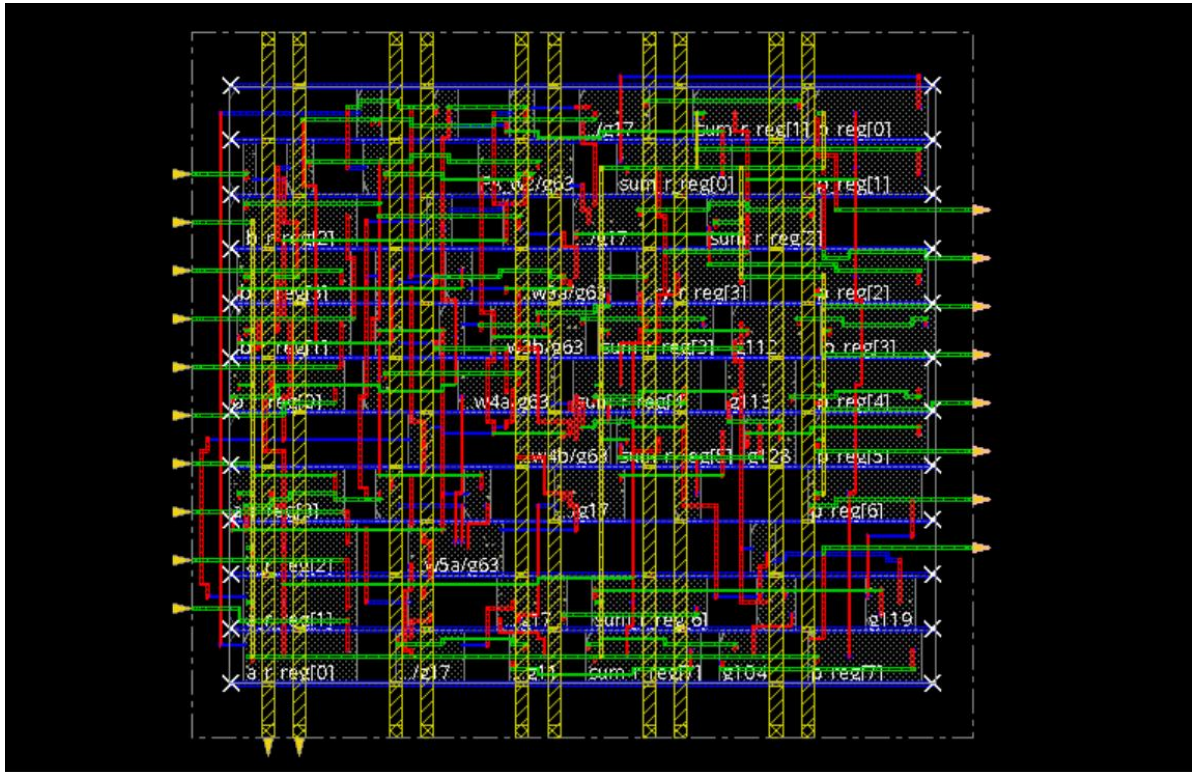


Figure23 : Implemented Power Stripe Layout

The implemented power grid is formed by regularly spaced vertical stripes extending across the entire core region. The uniform distribution of stripes ensures that power is delivered evenly to all standard cells, reducing the chances of localized voltage drop and improving overall reliability. The alternating pattern of VDD and VSS rails creates a balanced power network that supports stable operation.

The spacing and width choices result in a structure that avoids overlap and excessive density, leaving sufficient routing space for signal interconnections. The alignment of stripes from one boundary to the other confirms that the boundary settings were correctly applied. This configuration provides a strong and reliable power distribution network while maintaining good routability for subsequent placement and routing stages.

4.6.4 Cell Placement and Pre CTS Optimization



The standard cells are placed within the defined core area to achieve an organized and routable layout. Placement is performed by considering timing, wirelength, and congestion so that the cells are distributed efficiently without excessive overlap or unused dense regions.

At this stage, initial optimization is also performed before clock tree synthesis to improve placement quality and prepare the design for later CTS and routing steps. The visible trial routing helps estimate interconnect paths and identify congestion early, allowing the layout to be refined before clock insertion.

4.6.5 Adding Filler Cell and Metal Filling

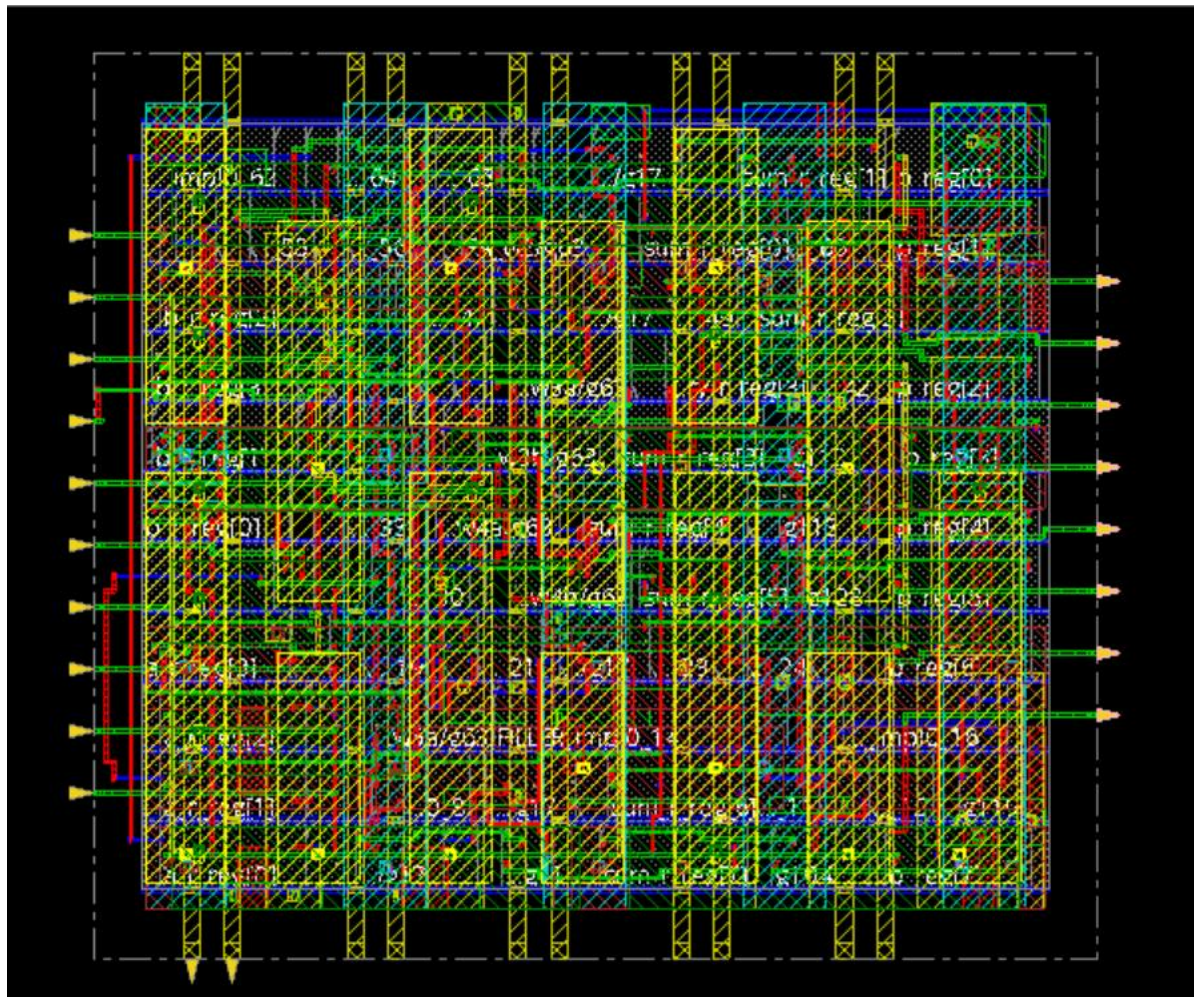


Figure 25: Layout After Metal Fill Insertion

The layout is updated by inserting metal fill structures across the design to satisfy density requirements and ensure manufacturability. Metal fill is added in empty regions of the layout without affecting circuit connectivity, helping maintain uniform metal distribution across all layers.

4.6.6 Exporting Data

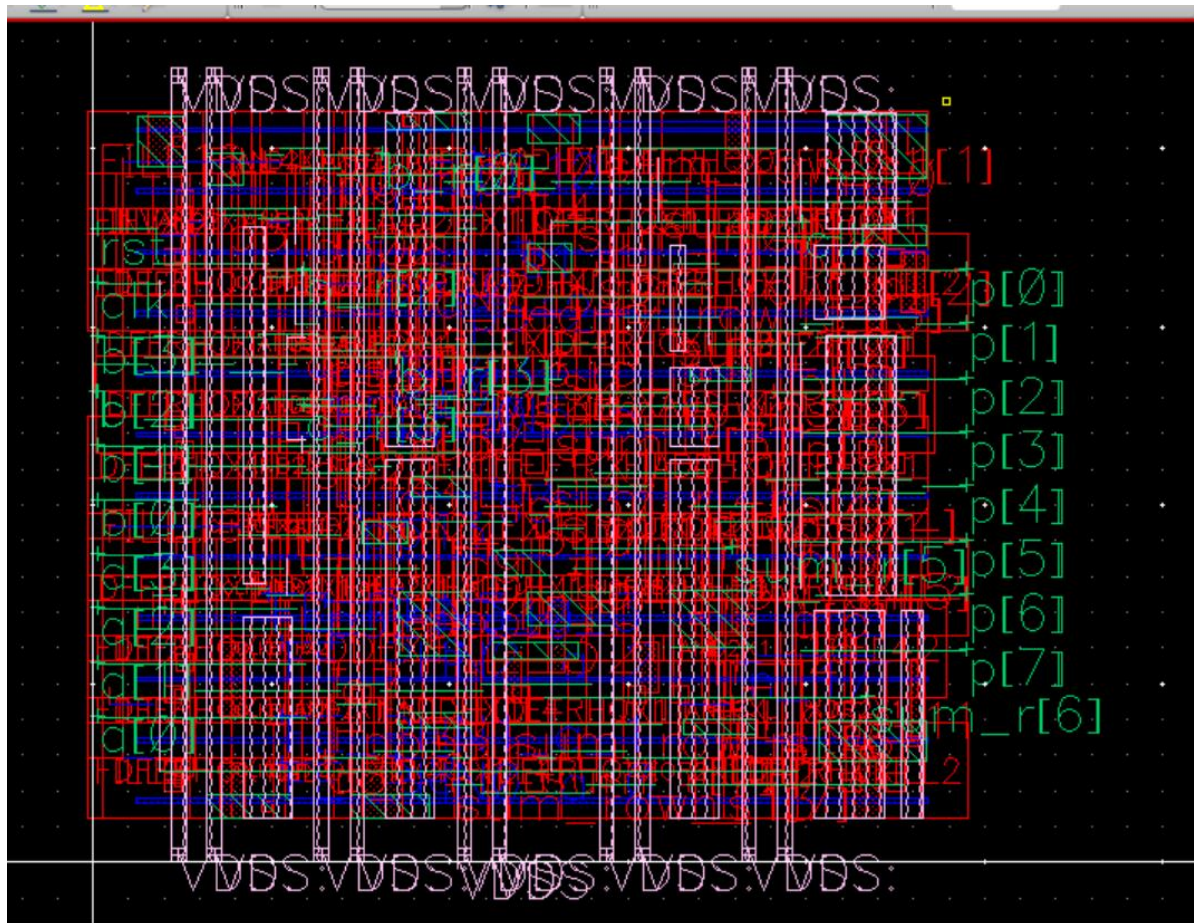
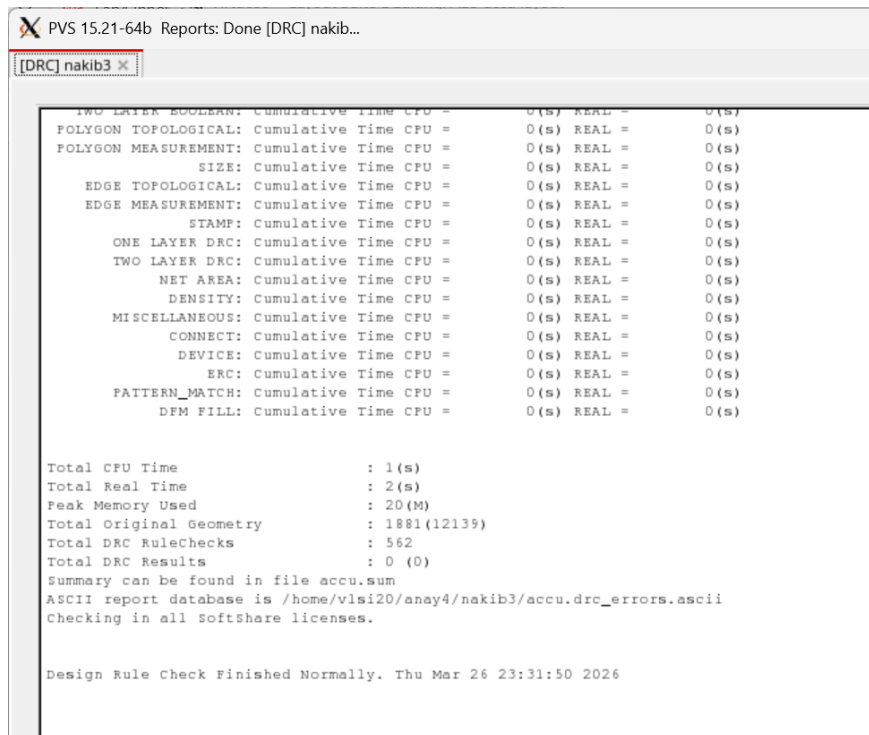


Figure 26: After importing GDS file in cadence virtuoso

4.6.7 Run DRC



```
PVS 15.21-64b Reports: Done [DRC] nakib...
[DRC] nakib3 X
TWO LAYER BOOLEAN: Cumulative Time CPU = 0(s) REAL = 0(s)
POLYGON TOPOLOGICAL: Cumulative Time CPU = 0(s) REAL = 0(s)
POLYGON MEASUREMENT: Cumulative Time CPU = 0(s) REAL = 0(s)
SIZE: Cumulative Time CPU = 0(s) REAL = 0(s)
EDGE TOPOLOGICAL: Cumulative Time CPU = 0(s) REAL = 0(s)
EDGE MEASUREMENT: Cumulative Time CPU = 0(s) REAL = 0(s)
STAMP: Cumulative Time CPU = 0(s) REAL = 0(s)
ONE LAYER DRC: Cumulative Time CPU = 0(s) REAL = 0(s)
TWO LAYER DRC: Cumulative Time CPU = 0(s) REAL = 0(s)
NET AREA: Cumulative Time CPU = 0(s) REAL = 0(s)
DENSITY: Cumulative Time CPU = 0(s) REAL = 0(s)
MISCELLANEOUS: Cumulative Time CPU = 0(s) REAL = 0(s)
CONNECT: Cumulative Time CPU = 0(s) REAL = 0(s)
DEVICE: Cumulative Time CPU = 0(s) REAL = 0(s)
ERC: Cumulative Time CPU = 0(s) REAL = 0(s)
PATTERN_MATCH: Cumulative Time CPU = 0(s) REAL = 0(s)
DFM FILL: Cumulative Time CPU = 0(s) REAL = 0(s)

Total CPU Time : 1(s)
Total Real Time : 2(s)
Peak Memory Used : 20(M)
Total Original Geometry : 1881(12139)
Total DRC RuleChecks : 562
Total DRC Results : 0(0)
Summary can be found in file accu.sum
ASCII report database is /home/vlsi20/anay4/nakib3/accu.drc_errors.ascii
Checking in all SoftShare licenses.

Design Rule Check Finished Normally. Thu Mar 26 23:31:50 2026
```

Figure 27 : DRC Verification Result

The design rule check (DRC) is performed after completing layout and metal fill insertion to verify compliance with fabrication constraints. The report shows that the total number of DRC violations is 000, indicating that all geometrical and spacing rules are satisfied. All rule checks are completed successfully without any errors, confirming that the layout meets the required design and manufacturing guidelines. This ensures that the design is ready for final sign-off and further stages such as tape-out or fabrication.

5 Design Analysis and Evaluation

5.1 Novelty

The main novelty of this project lies in dividing the multiplication process into two major functional segments and implementing them in a pipelined structure. In the proposed design, the first segment performs Wallace compression of the generated partial products, while the second segment performs the final summation of the reduced outputs. By inserting pipeline registers between these stages, the design is able to process input data in a staged manner, which improves the overall throughput of the multiplier compared to a purely combinational realization. Another important aspect of novelty is the treatment of the final summation stage. After Wallace reduction, the multiplication result is represented by intermediate sum and carry terms, and different summation approaches may be applied at this stage to optimize the final performance of the system. This separation of compression and summation provides greater design flexibility and creates scope for improving delay, timing performance, and implementation efficiency. Therefore, the proposed structure is not only a basic Wallace tree implementation, but also a more organized pipelined architecture that supports better operational flow and future optimization of the final addition stage.

5.2 Design Considerations (PO(c))

5.2.1 Considerations to public health and safety

Although this project is a digital circuit design, correctness and reliability are important for safe use in larger electronic systems. Therefore, the multiplier was verified carefully through functional testing, reset checking, and timing aware implementation to avoid incorrect operation.

5.2.2 Considerations to environment

Environmental consideration in this project is mainly related to power consumption and efficient hardware usage. By using a Wallace tree structure and analyzing power at different clock periods, the design supports a more efficient implementation with reduced unnecessary power loss.

5.2.3 Considerations to cultural and societal needs

Fast and reliable arithmetic circuits are essential for modern digital systems used in communication, computation, and automation. This project contributes to these needs by implementing a practical high speed multiplier architecture and developing skills relevant to real VLSI system design.

5.3 Investigations (PO(d))

This project does not involve physical experimentation; instead, the investigation is carried out through simulation, verification, and synthesis based analysis. Functional correctness is verified using directed and layered testbenches, including checks for pipeline behavior and reset operation. Performance characteristics such as timing and power are evaluated through

synthesis reports under different clock constraints. Based on these results, the effectiveness of the Wallace tree architecture and the proposed pipelined structure is analyzed in terms of correctness, throughput, and implementation feasibility.

5.4 Limitations of Tools (PO(e))

The entire work was completed using licensed Cadence tools, with Virtuoso used for overall design environment access, Genus for synthesis, and Innovus for PNR. Since all of these software tools are paid and license based, access to them is limited and sometimes difficult, which can restrict flexibility in experimentation and repeated runs. In addition, the reported timing and power results are tool estimated values and may differ slightly from actual fabricated hardware behavior.

6 Reflection on Individual and Team work (PO(i)).

6.1 Individual Contribution of Each Member

Student ID	Contribution
2006077	RTL code, Synthesis
2006096	RTL code, Directed Test Bench
2006091	Layered Test Bench
2006074,2006138	Physical Design

6.2 Mode of TeamWork

The project was completed through coordinated teamwork, where different members contributed to different stages such as RTL design, verification, synthesis, and physical implementation. Regular discussion and review among team members helped integrate the individual contributions into a complete final design.

6.3 Diversity Statement of Team

The team consisted of members with different strengths in coding, verification, analysis, and physical design, which helped distribute the project tasks effectively. This diversity of technical skills improved both the development process and the overall quality of the final implementation.

7 Communication to External Stakeholders (PO(j)).

7.1 Executive Summary

This project presents the design and physical implementation of a 4×4 Wallace tree multiplier using Verilog HDL and Cadence design tools. The proposed pipelined architecture improves throughput, while directed and layered verification confirm correct operation. Synthesis and physical design results show that the multiplier is functionally correct, timing-feasible, and suitable for high-speed digital arithmetic applications.

7.2 EDA code Link

- <https://edaplayground.com/x/YLYe>
- <https://edaplayground.com/x/HUPY>

8 Project Management and Cost Analysis (PO(k))

8.1 Timeline of Project Implementation

Week	Target Achieved
Week 3	Proposal Submitted & Approved
Week 5 & Week 6	RTL Code prepared
Week 7 & Week 8	Testbenches Prepared (Both directed and layered)
Week 9	Synthesis & optimization
Week 10 ,Week 11 & Week 12	PNR ,Optimization & Fixing DRC Error
Week 13	Final Project Presentation

9 Future Work (PO(I))

- The proposed design can be extended to higher bit-width multipliers such as 8×8 or 16×16 to study scalability in terms of delay, area, and power.
- Different fast adder architectures can be used in the final summation stage to further improve overall performance.
- The design can be implemented on FPGA or fabricated hardware in the future for practical validation beyond simulation and CAD based physical design.

10 References

- [1] A. D. Booth, "A Signed Binary Multiplication Technique," *The Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951.
- [2] M. Sjölander and P. Larsson-Edefors, "High-Speed and Low-Power Multipliers Using the Baugh-Wooley Algorithm and HPM Reduction Tree," *Proceedings of the IEEE International Conference on Electronics, Circuits and Systems*, 2008.
- [3] C. S. Wallace, "A Suggestion for a Fast Multiplier," *IEEE Transactions on Electronic Computers*, vol. EC-13, no. 1, pp. 14–17, 1964.
- [4] L. Dadda, "Some Schemes for Parallel Multipliers," *Alta Frequenza*, vol. 34, no. 5, pp. 349–356, 1965.
- [5] K. C. Bickerstaff, E. E. Swartzlander, Jr., and M. J. Schulte, "Analysis of Column Compression Multipliers," *Proceedings of the 15th IEEE Symposium on Computer Arithmetic*, pp. 33–39, 2001.
- [6] M. Janveja and V. Niranjana, "High Performance Wallace Tree Multiplier Using Improved Adder," *ICTACT Journal on Microelectronics*, vol. 3, no. 1, pp. 370–374, 2017.
- [7] P. Ramanathan, P. Anitha, and P. Kowsalya, "Modified Low Power DADDA Multiplier using Higher Order Compressors," *International Journal of Applied Engineering Research*, vol. 10, no. 9, 2015.
- [8] T. Yang, T. Ukezono, and T. Sato, "Low-Power and High-Speed Approximate Multiplier Design with a Tree Compressor," *Proceedings of the IEEE International Conference on Computer Design*, 2017.
- [9] <https://edaplayground.com/x/YLye>
- [10] <https://edaplayground.com/x/HUPY>